

Bachelorarbeit

im Studiengang

Informatik

zur Erlangung des Grades eines

Bachelor of Science (B.Sc.)

Thema:

**Hostbasierte Erkennung obfuskiertter
PowerShell-Aktivität
mittels ScriptBlock-Logging und Wazuh**

Ein Vergleich regelbasierter Wazuh-Erkennung und lexikalischer Merkmale
auf Basis von ScriptBlock-Logging unter Windows 11

vorgelegt von:

Rashed Alsuhaibi

geb. am 17.05.2001 in Dschidda, Saudi-Arabien

Matrikel-Nr.: 1459278

im Sommersemester 2026

im Studiengang Informatik

am Fachbereich Informatik und Ingenieurwissenschaften

der Frankfurt University of Applied Sciences

Bearbeitungsbeginn: 18.05.2026

Bearbeitungsende: 20.07.2026

Hauptreferent: Prof. Dr. Christian Baun

Korreferent: Prof. Dr. Thomas Gabel

Abgabedatum: 20.07.2026

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten und nicht veröffentlichten Quellen entnommen wurden, sind als solche kenntlich gemacht. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt.

Frankfurt am Main, 20.07.2026

Ort, Datum

A handwritten signature in black ink, consisting of a large, stylized 'F' with a horizontal bar and a vertical stroke, followed by a few additional strokes.

Unterschrift

Danksagung

Ich möchte mich bei allen Personen bedanken, die mich während der Erstellung dieser Bachelorarbeit fachlich und persönlich unterstützt haben. Mein besonderer Dank gilt Prof. Dr. Christian Baun für die fachliche Begleitung und die Rückmeldungen zur Eingrenzung des Themas sowie Prof. Dr. Thomas Gabel für die Übernahme des Korreferats. Ebenfalls danke ich Familie, Freunden und Kommilitoninnen und Kommilitonen für Motivation, Unterstützung und konstruktive Gespräche während der Bearbeitungszeit.

Zusammenfassung

Diese Bachelorarbeit untersucht die hostbasierte Erkennung obfuskiertter PowerShell-Aktivität unter Windows 11. Verglichen werden die unveränderten PowerShell-Standardregeln von Wazuh, ein nachvollziehbares lexikalisches Punktemodell und eine statistische Vergleichsbasis aus Zeichen- n -Grammen mit Term Frequency–Inverse Document Frequency (TF-IDF) und logistischer Regression. Beide lexikalischen Verfahren lesen den Feldwert `ScriptBlockText` direkt aus Event-ID 4104 des Kanals `Microsoft-Windows-PowerShell/Operational`. Wazuh verarbeitet dieselben Ereignisse parallel über Agent und Manager.

Der im Rahmen der Arbeit erzeugte und anschließend kuratierte Labordatensatz umfasst 300 ScriptBlocks. Er enthält 150 benigne Fälle und 150 simulierte Testfälle mit Angriffsindikatoren aus 30 definierten Verhaltensmustern. Öffentliche Quellen dienen nur zur Auswahl und Einordnung der Techniken; vollständige C2-Implantate oder Payload-Dateien wurden nicht übernommen. Aus den 30 Grundmustern entstanden 30 klare und 120 obfuskierte positive Varianten. Sie geben ausschließlich inerte Texte aus und führen weder Netzwerkzugriffe noch Persistenz- oder Schutzfunktionsänderungen aus. Der ausgeglichene, vom Verfasser gelabelte Datensatz bildet keine betriebliche Klassenverteilung ab.

Im kontrollierten Messlauf wurden alle 300 Testfälle lokal erfasst. Eine getrennte managerseitige Prüfung bestätigte zusätzlich den Empfang aller 300 Ereignisse durch Wazuh; 88 davon führten zu einer Alarmmeldung der Standardregeln. Die lexikalischen Verfahren wurden in einer fünffachen, nach Verhaltensfamilien gruppierten Kreuzvalidierung bewertet. Wazuh erreichte eine Precision von 0,6136, einen Recall von 0,3600, einen F1-Score von 0,4538 und eine False-Positive-Rate von 0,2267. Das Punktemodell erreichte 0,8760, 0,7533, 0,8100 und 0,1067. Die statistische Vergleichsbasis erreichte 0,9060, 0,9000, 0,9030 und 0,0933. Eine isolierte Laufzeitmessung maß für das Punktemodell bei 300 Auswertungen im Mittel 1,661 Millisekunden je ScriptBlock.

Die Ergebnisse zeigen innerhalb des untersuchten Datensatzes einen Nutzen lexikalischer Textmerkmale. Wegen der sicheren Testfallkonstruktion, des einzelnen Testhosts und der fehlenden Prüfung mit einem unabhängigen externen Datensatz erlauben sie jedoch keine Aussage über die Leistung in Produktivumgebungen.

Abstract

This bachelor thesis investigates host-based detection of obfuscated PowerShell activity on Windows 11. It compares the unchanged Wazuh standard rules for PowerShell, an interpretable lexical scoring model, and a statistical baseline using term frequency-inverse document frequency (TF-IDF) representations of character n -grams and logistic regression. Both lexical methods read `ScriptBlockText` directly from event ID 4104 in the `Microsoft-Windows-PowerShell/Operational` channel. Wazuh processes the same events in parallel through its agent and manager.

The synthetic laboratory dataset was created and curated as part of this work. It contains 150 benign cases and 150 simulated indicator cases from 30 defined behavior patterns. Public sources were used only to select and classify the represented techniques; no complete C2 implants or payload files were imported. The 30 patterns produced 30 clear and 120 obfuscated positive variants. They only emit inert text and perform no network access, persistence, or security-control modification. The author-labeled dataset is balanced and does not represent an operational class distribution.

During the controlled run, all 300 test cases were captured locally. A separate manager-side verification also confirmed receipt of all 300 events by Wazuh; 88 produced a standard-rule alert. The lexical methods were evaluated using five-fold grouped cross-validation, keeping each behavior family in one fold. Wazuh achieved a precision of 0.6136, recall of 0.3600, F1-score of 0.4538, and false-positive rate of 0.2267. The scoring model achieved 0.8760, 0.7533, 0.8100, and 0.1067. The statistical baseline achieved 0.9060, 0.9000, 0.9030, and 0.0933. A bounded runtime experiment measured a mean of 1.661 milliseconds per script block for the scoring model over 300 evaluations.

Within the examined dataset, the results show that lexical text features provide additional detection value. The inert test construction, single test host, and absence of an independent external evaluation prevent conclusions about performance in production environments.

Inhaltsverzeichnis

Eidesstattliche Erklärung	ii
Danksagung	iii
Zusammenfassung	iv
Abstract	v
Abbildungsverzeichnis	viii
Tabellenverzeichnis	ix
Quell- und Pseudocodeverzeichnis	x
Abkürzungsverzeichnis	xi
1 Einleitung	1
1.1 Motivation und Hintergrund	1
1.2 Problemstellung	1
1.3 Zielsetzung der Arbeit	2
1.4 Forschungsfragen	2
1.5 Abgrenzung und Sicherheitsrahmen	3
1.6 Vorgehensweise	3
1.7 Aufbau der Arbeit	4
2 Grundlagen	5
2.1 PowerShell als Administrations- und Automatisierungsumgebung	5
2.2 ScriptBlock-Logging	5
2.3 PowerShell-Obfuskierung	6
2.4 Hostbasierte Angriffserkennung	6
2.5 Wazuh	6
2.6 Lexikalische Merkmale	7
3 Stand der Forschung und verwandte Ansätze	8
3.1 Regel- und signaturbasierte Erkennung	8
3.2 Lernbasierte Klassifikation von PowerShell-Code	8
3.3 Deobfuskation und semantische Analyse	9
3.4 Vergleich der Ansätze	9
3.5 Forschungslücke und eigener Beitrag	10
4 Methodik und Versuchsdesign	12
4.1 Anforderungen und Entwurfsentscheidungen	12
4.2 Untersuchungsdesign	14
4.3 Reproduzierbarkeit und Sicherheitsanforderungen	14
4.4 Lab-Architektur	15
4.5 Datensatzdesign	16
4.6 Klassen und Testfälle	18
4.7 Sicherheitskonzept der simulierten Testfälle	19
4.8 Lexikalische Merkmale und Punktmodell	19
4.9 Statistische lexikalische Vergleichsbasis	20
4.10 Wazuh-Referenzmessung und Alarmzuordnung	21
4.11 Evaluierungsmetriken	22
4.12 Schwellenwertwahl und Kreuzvalidierung	22

5	Implementierung	24
5.1	Aktivierung und Prüfung des ScriptBlock-Loggings	24
5.2	Wazuh-Agent und Ereigniskanal-Weiterleitung	25
5.3	Ausführung der Datensatz-Testfälle	26
5.4	Managerseitiger Empfangsnachweis und Alarmexport	26
5.5	Python-Pipeline	28
5.6	Qualitätssicherung der Artefakte	30
6	Evaluation	31
6.1	Datenbasis des Laborlaufs	31
6.2	Schwellenwertanalyse	31
6.3	Wazuh-Standardregeln	32
6.4	Statistische lexikalische Vergleichsbasis	33
6.5	Gesamtvergleich der Erkennungsverfahren	34
6.6	Gruppierte Kreuzvalidierung	36
6.7	Auswertung nach Obfuskierungsart	37
6.8	Bewertung der False Positives	39
6.9	Qualitative Fehleranalyse	40
6.10	Laufzeit des Punktemodells	41
7	Diskussion	42
7.1	Interpretation der Ergebnisse	42
7.2	Einordnung der Wazuh-Referenzmessung	42
7.3	Nutzen und Grenzen der lexikalischen Verfahren	43
7.4	Interne Validität	43
7.5	Übertragbarkeit und Grenzen der Messung	44
7.6	Herausforderungen während der Umsetzung	44
7.7	Praktische Konsequenzen	45
8	Fazit und Ausblick	46
8.1	Zusammenfassung	46
8.2	Beantwortung der Forschungsfragen	46
8.3	Bewertung der Zielerreichung	47
8.4	Ausblick	47
	Literaturverzeichnis	49
A	Datensatzstruktur	53
B	Reproduzierbarkeit und Artefakte	54
	Dokumentation der verwendeten Hilfsmittel	57
	Quellcode und GitHub-Repository	58

Abbildungsverzeichnis

4.1	Laborarchitektur mit getrennten Erkennungs- und Nachweispfaden . .	15
4.2	Konstruktion des synthetischen Labordatensatzes und Rolle der Quellen	17
4.3	Versuchsablauf von der Ausführung bis zur gemeinsamen Auswertung	23
5.1	Aktivierter Registry-Wert für PowerShell ScriptBlock-Logging	25
5.2	Event-ID 4104 mit Testfall-ID im ScriptBlockText	25
5.3	Aktiver Windows-Agent mit Agent-ID, Hostname und Wazuh-Version	26
5.4	Wazuh-Alarmmeldung mit Regel-ID, Event-ID 4104 und Testfall-ID .	27
5.5	JSONL-Alarmexport mit Regelmetadaten und ScriptBlockText	28
5.6	Direkter Datenfluss vom Windows-Ereigniskanal zu den lexikalischen Verfahren	29
5.7	Abgeschlossener Pipeline-Lauf mit Eingaben und Ergebnisverzeichnis	30
6.1	Kennzahlen des Punktemodells in Abhängigkeit vom Schwellenwert .	32
6.2	Vergleich der drei Einzelverfahren	35
6.3	Gruppenstratifizierte Bootstrap-Intervalle für F1-Score und FPR . . .	36
6.4	Recall von Wazuh und Punktemodell nach Obfuskierungsart	38
6.5	Durchsatz und Latenz des Punktemodells in der isolierten Laufzeit- messung	41

Tabellenverzeichnis

2.1	Betrachtete PowerShell-Obfuskierungstechniken	6
3.1	Vergleich ausgewählter Ansätze zur PowerShell-Erkennung	10
4.1	Anforderungen, Prüfkriterien und Nachweise	13
4.2	Technische Testumgebung und verwendete Versionen	16
4.3	Klassen und Gruppen des Datensatzes	18
4.4	Obfuskierungsarten im Datensatz	19
4.5	Punktregeln des lexikalischen Detektors	20
6.1	Entscheidungsrelevanter Bereich der Schwellenwertanalyse	32
6.2	Ausgelöste Wazuh-Regeln im Alarmexport	33
6.3	Sensitivität gegenüber der Wazuh-Mindestregelstufe	33
6.4	Out-of-Fold-Ergebnisse der statistischen Vergleichsbasis	34
6.5	Gesamtvergleich der Entscheidungen für 300 Testfälle	35
6.6	Out-of-Fold-Ergebnisse des Punktemodells	37
6.7	Recall der 120 obfuskerten Testfälle	38
6.8	False Positives nach benigner Datensatzgruppe	39
6.9	Erwartete Precision bei angenommener positiver Prävalenz	39
6.10	Repräsentative Einzelentscheidungen der drei Verfahren	40
6.11	Laufzeitmessung der Merkmalsextraktion und Punktbewertung	41
B.1	Prüfsummen der zentralen Versuchsartefakte	54

Quell- und Pseudocodeverzeichnis

5.1	Aktivierung des ScriptBlock-Loggings	24
5.2	Wazuh-Agent-Konfiguration für den PowerShell-Ereigniskanal	25
5.3	Aufruf des zentralen Methodenvergleichs	29
5.4	Aufruf der statistischen Vergleichsbasis	29
A.1	Vereinfachte Datensatzstruktur	53

Abkürzungsverzeichnis

Abkürzung	Bedeutung
AMSI	Antimalware Scan Interface
AST	Abstract Syntax Tree
C2	Command and Control
F1	F1-Score, harmonisches Mittel aus Precision und Recall
FN / FP	False Negative / False Positive
FPR	False-Positive-Rate
TF-IDF	Term Frequency–Inverse Document Frequency
JSON / JSONL	JavaScript Object Notation / zeilenorientiertes JSON
LOLBins	Living-off-the-Land Binaries
TN / TP	True Negative / True Positive
WSL	Windows Subsystem for Linux
XML	Extensible Markup Language

1 Einleitung

Dieses Kapitel führt in die untersuchte Erkennungsaufgabe ein. Es beschreibt Motivation und Problemstellung, legt Ziele und Forschungsfragen fest und grenzt den Sicherheits- und Geltungsbereich der Arbeit ab.

1.1 Motivation und Hintergrund

PowerShell ist unter Windows ein Werkzeug für Administration, Automatisierung und Systemverwaltung. Das MITRE-ATT&CK-Framework führt ihre missbräuchliche Nutzung als Technik zur Befehls- und Skriptausführung [5]. Administrative Skripte konfigurieren Systeme, werten Ereignisse aus und steuern Dienste. Dieselben Funktionen können auch für Angriffe eingesetzt werden. Dabei lassen sich Befehle dynamisch zusammensetzen, kodieren oder ohne zusätzliche ausführbare Datei starten. Die bloße Nutzung von PowerShell ist deshalb kein ausreichender Angriffsindikator.

Obfuskierung verändert die textuelle Form eines Skripts, ohne dessen beabsichtigte Funktion zwingend zu ändern. Kodierung, String-Konkatenation, Backticks oder wechselnde Groß- und Kleinschreibung können feste Suchmuster umgehen. PowerShell ScriptBlock-Logging stellt mit Event-ID 4104 den interpretierten Skriptinhalt als hostbasierte Telemetrie bereit [1, 2]. Diese Arbeit untersucht, wie die installierten Wazuh-Standardregeln und zwei lexikalische Verfahren auf dieser Datenquelle entscheiden.

1.2 Problemstellung

Regelbasierte Erkennung ordnet eine Alarmmeldung einem dokumentierten Muster zu. Diese Nachvollziehbarkeit ist nützlich, doch die Erkennung hängt von der beobachteten Schreibweise ab. Verändert Obfuskierung die erwarteten Tokens, bleibt eine Regel möglicherweise ohne Treffer. Breite Regeln können dagegen legitime Verwaltungsbefehle alarmieren.

Der Vergleich benötigt daher identische Testfälle, eine nachgewiesene Telemetrieübertragung und getrennte Entscheidungen. Die Wazuh-Referenz bleibt unverändert. Als lexikalische Gegenstücke dienen ein erklärbares Punktemodell sowie eine statistische

Vergleichsbasis aus Zeichen- n -Gramm-TF-IDF und logistischer Regression. Zwei binäre Verknüpfungen untersuchen ergänzend, ob sich Wazuh und Punktemodell sinnvoll kombinieren lassen.

1.3 Zielsetzung der Arbeit

Ziel ist die Konzeption, Implementierung und Auswertung eines hostbasierten Erkennungs- und Evaluierungssystems für obfuskierte PowerShell-Aktivität unter Windows 11. Die Untersuchung verfolgt folgende Teilziele:

- Z1 Aufbau eines kontrollierten Windows-11-Labors mit ScriptBlock-Logging und Wazuh-Anbindung.
- Z2 Erstellung eines dokumentierten, synthetischen Labordatensatzes mit benignen und simulierten ScriptBlocks sowie kontrollierten Obfuskationsvarianten.
- Z3 Messung der unveränderten Wazuh-Standardregeln einschließlich eines managerseitigen Empfangsnachweises.
- Z4 Implementierung eines lexikalischen Punktemodells mit direktem Event-4104-Eingang.
- Z5 Implementierung einer statistischen lexikalischen Vergleichsbasis unter denselben gruppierten Teilstichproben.
- Z6 Vergleich anhand von Precision, Recall, F1-Score, False-Positive-Rate und einer isolierten Laufzeitmessung.

1.4 Forschungsfragen

Die zentrale Forschungsfrage lautet:

Welche Erkennungsleistung zeigen unveränderte Wazuh-Standardregeln und lexikalische Verfahren bei obfuskierten PowerShell-ScriptBlocks im festgelegten Laborversuch?

Daraus folgen fünf Teilfragen:

- F1 Welche obfuskierten Testfälle lösen eine Wazuh-Standardregel aus?

- F2 Welche Merkmale und Rekonstruktionsschritte tragen zum Ergebnis des lexikalischen Punktemodells bei?
- F3 Wie unterscheiden sich Punktemodell und statistische Zeichen- n -Gramm-Vergleichsbasis?
- F4 Wie verändern ODER- und UND-Verknüpfung von Wazuh und Punktemodell die Kennzahlen?
- F5 Bei welchen benignen Gruppen und Obfuskierungsarten entstehen Fehlentscheidungen?

1.5 Abgrenzung und Sicherheitsrahmen

Alle Experimente finden in einer lokalen Laborumgebung statt. C2-nahe und andere schädlich wirkende Muster werden ausschließlich als nicht ausgeführte Zeichenketten dargestellt. Es werden weder externe C2-Verbindungen hergestellt noch funktionsfähige Schadsoftware oder nachgeladene Nutzlasten ausgeführt. Netzwerkindikatoren verwenden reservierte `.invalid`-Domänen und IANA-TEST-NET-Adressen gemäß RFC 2606 und RFC 5737 [22, 23].

Die Untersuchung beschränkt sich auf Event-ID 4104, Windows PowerShell 5.1, einen Windows-11-Host und die installierte Wazuh-Regelbasis. Sie bewertet keine vollständige Endpoint-Detection-and-Response-Lösung. Organisationsspezifische Skripte, reale Schadsoftwareausführung und unabhängige Zweitlabels liegen nicht vor.

1.6 Vorgehensweise

Zunächst werden Datensatz, Sicherheitsprüfungen und Laboraufbau festgelegt. Ein kontrollierter Lauf erzeugt Event-4104-Telemetrie, die lokal und managerseitig auf Vollständigkeit geprüft wird. Danach werden Wazuh-Alarme, das lexikalische Punktemodell und die statistische Vergleichsbasis mit denselben IDs und Labels ausgewertet. Gruppierte Kreuzvalidierung trennt verwandte Verhaltensfamilien zwischen Trainings- und Testteilstichproben. Bootstrap-Intervalle, Prävalenzrechnung, Fehleranalyse und Laufzeitmessung ergänzen die Punktkennzahlen.

1.7 Aufbau der Arbeit

Kapitel 2 erläutert PowerShell, ScriptBlock-Logging, Obfuskierung, Wazuh und lexikalische Merkmale. Kapitel 3 vergleicht verwandte regelbasierte, lernbasierte und deobfusierende Ansätze. Kapitel 4 beschreibt Versuchsdesign, Datensatz und Bewertungsverfahren. Kapitel 5 dokumentiert die Implementierung und die Nachweise der Datenerfassung. Kapitel 6 enthält die Messergebnisse. Kapitel 7 diskutiert Aussagekraft und Grenzen. Kapitel 8 beantwortet die Forschungsfragen und schließt mit einem Ausblick.

2 Grundlagen

Dieses Kapitel erläutert die technischen Begriffe und Datenflüsse des Versuchs. Der Schwerpunkt liegt auf PowerShell, Event-ID 4104, Obfuskierung, hostbasierter Erkennung, Wazuh und textbasierten Merkmalen.

2.1 PowerShell als Administrations- und Automatisierungsumgebung

PowerShell verbindet Kommandozeile, Skriptsprache und Zugriff auf .NET sowie Windows-Verwaltungsschnittstellen. Skripte können Dateien, Prozesse, Dienste, Registry, Netzwerkfunktionen und Systeminformationen verarbeiten. Da PowerShell auf Windows-Systemen vorhanden ist, lässt sie sich auch im Sinne des Living-off-the-Land-Prinzips missbrauchen. Für die Erkennung sind daher Inhalt und Kontext eines ScriptBlocks aussagekräftiger als der Prozessname allein.

2.2 ScriptBlock-Logging

PowerShell ScriptBlock-Logging speichert ausgeführte ScriptBlocks im Kanal `Microsoft-Windows-PowerShell/Operational` [1, 2]. Event-ID 4104 enthält den Feldwert `ScriptBlockText`. Dieser kann mehr Inhalt bereitstellen als eine reine Prozessstartmeldung. Lange ScriptBlocks können in mehreren Fragmenten auftreten; deshalb muss eine Erfassung Fragmentierung, Ereignis-ID und vollständigen Text prüfen.

Im Versuchsaufbau lesen zwei Pfade denselben Kanal. Ein lokaler Erfasser schreibt `ScriptBlockText` für die lexikalische Analyse in eine JSONL-Datei. Der Wazuh-Agent überträgt die Ereignisse parallel an den Manager. Damit bleibt die Texteingabe der lexikalischen Verfahren von der Wazuh-Alarmbildung getrennt.

2.3 PowerShell-Obfuskierung

Obfuskierung verändert die Darstellung eines Skripts, um seine Analyse zu erschweren. Tabelle 2.1 fasst die im Datensatz betrachteten Kategorien zusammen.

Tabelle 2.1: Betrachtete PowerShell-Obfuskierungstechniken

Technik	Auswirkung auf die Textdarstellung
Base64-Kodierung	Ersetzt lesbaren Text durch eine kodierte Zeichenfolge, die vor einer Inhaltsprüfung dekodiert werden muss.
String-Konkatenation	Teilt Tokens auf mehrere Literale auf und setzt sie erst durch Operatoren oder Methoden zusammen.
Backticks	Unterbrechen oder maskieren Zeichenfolgen mit einem PowerShell-spezifischen Escape-Zeichen.
Gemischte Schreibweise	Variiert Groß- und Kleinschreibung in Befehlen und Indikatoren.
Alias- und Befehlsrekonstruktion	Ersetzt ausgeschriebene Cmdlets durch Aliase oder baut Befehle aus Fragmenten auf.
Kombinierte Obfuskation	Verwendet mindestens zwei der genannten Veränderungen im selben ScriptBlock.

2.4 Hostbasierte Angriffserkennung

Hostbasierte Systeme zur Angriffserkennung werten Ereignisse eines Endsystems aus. Dazu gehören Protokolle, Prozesse, Konfigurations- und Dateiänderungen. NIST ordnet hostbasierte Verfahren als eine Kategorie von Intrusion-Detection- und Intrusion-Prevention-Systemen ein [6]. Für PowerShell ist der Endpunkt relevant, weil dort der interpretierte Skriptinhalt vorliegt, auch wenn Netzwerkverkehr verschlüsselt ist oder keine zusätzliche Datei entsteht.

2.5 Wazuh

Wazuh ist eine quelloffene Plattform zur Sicherheitsüberwachung. Ein Endpunktagent sammelt Ereignisse, der Manager wertet sie mit Decodern und XML-Regeln aus

und schreibt passende Alarmmeldungen [3, 4]. Im Versuch sammelt der Agent den PowerShell-Ereigniskanal. Die installierten PowerShell-Standardregeln bilden die regelbasierte Referenz. Eine Alarmmeldung enthält unter anderem Regel-ID, Regelstufe, Agent und Ereignisdaten.

Das Ereignisarchiv des Managers und das Alarmprotokoll erfüllen verschiedene Aufgaben. Das Archiv kann den Eingang eines Ereignisses belegen, auch wenn keine Regel auslöst. Das Alarmprotokoll enthält dagegen nur Ereignisse mit Regelentscheidung. Für die lexikalische Analyse wird keines dieser beiden Wazuh-Protokolle als Texteingabe verwendet.

2.6 Lexikalische Merkmale

Lexikalische Merkmale beschreiben die textuelle Form eines ScriptBlocks. Zum Punktemodell gehören beispielsweise Länge, Backticks, Stringlitterale, Kodierungsindikatoren und Treffer dokumentierter Tokenkategorien. Die Shannon-Entropie beschreibt die Verteilung der Zeichen [7]. Ein hoher oder niedriger Wert allein belegt jedoch keine Schädlichkeit.

Zeichen- n -Gramme zerlegen einen Text in überlappende Zeichenfolgen fester Länge. Term Frequency–Inverse Document Frequency (TF-IDF) gewichtet häufige Fragmente innerhalb eines Dokuments und reduziert den Einfluss von Fragmenten, die in vielen Trainingsdokumenten vorkommen [8]. Eine logistische Regression kann daraus eine Wahrscheinlichkeit für die positive Klasse bestimmen. Im Gegensatz zum festen Punktemodell lernt sie ihre Gewichte aus Trainingsdaten; ihre Auswertung muss deshalb die Trennung von Training und Test sicherstellen.

3 Stand der Forschung und verwandte Ansätze

Dieses Kapitel ordnet die Arbeit anhand von vier Kriterien ein: Telemetriequelle, Text- oder Code-Repräsentation, Entscheidungsverfahren und Validierungsgrundlage. Die Kriterien machen sichtbar, welche Aufgaben bestehende Ansätze lösen und welche Abgrenzung für den vorliegenden Laborvergleich gilt.

3.1 Regel- und signaturbasierte Erkennung

Regelbasierte Systeme prüfen Telemetrie auf Zeichenketten, reguläre Ausdrücke, Feldkombinationen oder Ereignisfolgen. Wazuh verwendet Decoder und XML-Regeln; Sigma beschreibt Erkennungslogik herstellerunabhängig [4, 10]. Eine Alarmmeldung lässt sich auf eine konkrete Regel zurückführen. Die Erkennung hängt jedoch von der formulierten Oberfläche ab. Kodierung oder Rekonstruktion kann ein erwartetes Muster verändern, während breite Regeln auch legitime Administration erfassen.

Die vorliegende Referenzmessung übernimmt die installierten Wazuh-Regeln unverändert. Zusätzlich wird die Mindestregelstufe nachträglich variiert. Diese Sensitivitätsanalyse zeigt, wie sich der Betriebspunkt innerhalb derselben Regelbasis verschiebt, ohne daraus eine auf den Datensatz angepasste neue Regelbasis abzuleiten.

3.2 Lernbasierte Klassifikation von PowerShell-Code

Hendler, Kels und Rubin vergleichen klassische Merkmale mit zeichenbasierten neuronalen Netzen und untersuchen ein Ensemble zur Erkennung schädlicher PowerShell-Befehle [13]. Rusak, Al-Dujaili und O'Reilly verwenden Strukturen des Abstract Syntax Tree (AST) als Eingabe eines tiefen Lernverfahrens [14]. Rubin, Kels und Hendler analysieren über die Antimalware Scan Interface (AMSI) erfassten Code mit kontextuellen Zeichen- und Tokenrepräsentationen [15].

Hung, Chen und Ma kombinieren 33 statische Text- und Verhaltensmerkmale mit XGBoost. Nach der Vorverarbeitung umfasst ihr Datensatz 8.197 Skripte aus getrennten benignen und schädlichen Quellen [16]. Wu et al. koppeln in MPSD die Erfassung über AMSI mit einer Bewertung durch VirusTotal. Die veröffentlichte Untersuchung verwendet 82 benigne und 218 schädliche Skripte; die externe Abfrage verursacht laut Messung im Mittel etwa 50 Sekunden Verzögerung je Analyse [17]. Beide Arbeiten zeigen, dass Datengrundlage, Erfassungsstelle und Betriebsziel die Vergleichbarkeit von Kennzahlen bestimmen.

Die statistische Vergleichsbasis dieser Arbeit verwendet Zeichen- n -Gramm-TF-IDF mit logistischer Regression. Sie dient als algorithmische Kontrolle für das feste Punktemodell. Ihre Werte werden ausschließlich innerhalb des eigenen Datensatzes interpretiert.

3.3 Deobfuskation und semantische Analyse

Deobfuskationsverfahren versuchen, eine leichter auswertbare Darstellung wiederherzustellen. PSDEM verbindet statische und dynamische Schritte [18]. PowerDrive dokumentiert eine mehrstufige Analyse von PowerShell-Schadsoftware [19]. Li et al. kombinieren AST-Teilbäume, Emulation und semantische Signaturen [20]. PowerPeeper rekonstruiert Ausführungszustände dynamisch auf Instruktionsebene und wurde 2024 auf der ACM CCS veröffentlicht [21].

Diese Verfahren erfassen Syntax, Datenfluss oder Laufzeitverhalten und gehen damit über eine Textklassifikation hinaus. Sie benötigen zugleich Parser-, Emulations- oder Ausführungslogik. Das Punktemodell der vorliegenden Arbeit rekonstruiert lediglich begrenzte Zeichenketten und dekodiert geeignete Base64-Kandidaten, ohne den rekonstruierten Inhalt auszuführen.

3.4 Vergleich der Ansätze

Tabelle 3.1 stellt ausgewählte Ansätze anhand der genannten Kriterien gegenüber. Die publizierten Kennzahlen werden nicht direkt übernommen, weil Datensätze, Klassenverteilungen und Telemetriequellen voneinander abweichen.

Tabelle 3.1: Vergleich ausgewählter Ansätze zur PowerShell-Erkennung

Ansatz	Eingabe und Verfahren	Abgrenzung zur vorliegenden Arbeit
Wazuh und Sigma [4, 10]	Ereignisfelder und dokumentierte Muster.	Regelbasiert und erklärbar; Wazuh bildet die unveränderte Referenzmessung.
Hendler et al. [13]	Merkmale, Zeichenmodell und Ensemble auf PowerShell-Befehlen.	Lernbasierte Gewichte und externe Datengrundlage.
Rusak et al. [14]	AST-Repräsentation und tiefes Lernverfahren.	Verarbeitet Syntaxstruktur statt ausschließlich Textmerkmale.
Rubin et al. [15]	AMSI-Telemetrie und kontextuelle Einbettungen.	Andere Erfassungsstelle und höhere Modellkomplexität.
Hung et al. [16]	33 statische Merkmale und XGBoost auf 8.197 vorverarbeiteten Skripten.	Verwandter Merkmalsansatz, aber andere Quellen, Labels und Stichprobengröße.
Wu et al. [17]	AMSI-Erfassung und externe Mehrfachbewertung über VirusTotal.	Präventives Blockieren statt nachgelagerter Event-4104-Auswertung.
PSDEM, PowerDrive und Li et al. [18, 19, 20]	Statische und dynamische Deobfuskation sowie semantische Signaturen.	Größere Analysetiefe als die begrenzte Textrekonstruktion.
PowerPeeler [21]	Dynamische Rekonstruktion auf Instruktionsebene.	Berücksichtigt Laufzeitsemantik, die im Laborversuch nicht erfasst wird.

3.5 Forschungslücke und eigener Beitrag

Bestehende Arbeiten untersuchen regelbasierte Erkennung, statistische Klassifikation und Deobfuskation meist auf anderen Telemetriequellen oder Datensätzen. Für die konkrete Kombination aus Event-ID 4104, unveränderten Wazuh-Standardregeln,

einem erklärbaren Punktemodell und einem statistischen lexikalischen Kontrollverfahren liegt damit kein direkt übertragbarer Messwert vor.

Der eigene Beitrag besteht aus der Definition und sicheren Erzeugung des Datensatzes, dem getrennten lokalen und managerseitigen Telemetrienachweis, der eindeutigen Alarmzuordnung, den beiden implementierten lexikalischen Verfahren und ihrer gemeinsamen gruppierten Auswertung. Der Beitrag ist ein reproduzierbarer Laborvergleich, kein allgemein trainierter Schadsoftwareklassifikator.

4 Methodik und Versuchsdesign

Dieses Kapitel legt fest, wie die Forschungsfragen untersucht werden. Es beschreibt Anforderungen, Untersuchungseinheit, getrennte Datenpfade, Datensatz, Erkennungsverfahren und statistische Auswertung.

4.1 Anforderungen und Entwurfsentscheidungen

Tabelle 4.1 leitet aus den Forschungsfragen überprüfbare Anforderungen ab. Die Nachweise verweisen auf maschinenlesbare Artefakte statt auf den bloßen Zustand einer Benutzeroberfläche.

Tabelle 4.1: Anforderungen, Prüfkriterien und Nachweise

Anforderung	Prüfkriterium	Umsetzung und Nachweis
Lokale Telemetrie	Für jeden ausgeführten Testfall liegt genau ein vollständiges Event 4104 vor.	Abgleich von Ausführungslog, ID, Event-ID und <code>ScriptBlockText</code> für 300 Testfälle.
Wazuh-Transport	Auch Testfälle ohne Alarm müssen den Manager erreicht haben.	Getrennter Ausschnitt des managerseitigen Ereignisarchivs mit 300 eindeutigen IDs.
Vergleichbarkeit	Alle Verfahren verwenden dieselben Testfall-IDs und Klassenlabels.	Direkter Windows-Export für beide Lexikalverfahren; getrennte Wazuh-Alarmzuordnung über neutrale IDs.
Trennung von Training und Test	Verwandte Varianten dürfen nicht beide Datenanteile belegen.	Fünffache, nach Verhaltensfamilien gruppierte Out-of-Fold-Auswertung.
Erklärbarkeit	Entscheidungen des Punktemodells müssen prüfbar sein.	Ausgabe von Merkmalen, Punktbeiträgen, Schwellenwert und Klassifikation je Testfall.
Sicherheit	Positive Testfälle dürfen keine schädliche Wirkung entfalten.	Inerte Texte, reservierte Netzindikatoren, AST-Prüfung und eingeschränkte Befehlsliste.
Reproduzierbarkeit	Eingaben, Programme und Ergebnisse müssen eindeutig identifizierbar sein.	Feste Parameter, maschinenlesbare Ausgaben, Tests und SHA-256-Prüfsummen.
Rechenaufwand	Die isolierte Textauswertung muss messbar sein.	Laufzeitversuch mit 300, 1.500 und 3.000 Auswertungen ohne Ein-/Ausgabe und Wazuh-Verarbeitung.

4.2 Untersuchungsdesign

Untersuchungseinheit ist ein ausgeführter Testfall. Sein `ScriptBlockText` wird direkt aus Event-ID 4104 geladen und über eine neutrale ID dem Datensatzlabel zugeordnet. Das Label `benign` bildet die negative Klasse; `simulated_malicious` bildet die positive Klasse. Das positive Label bezeichnet einen Testfall mit einem zuvor definierten Angriffsindikator. Es bezeichnet weder eine schädliche Wirkung noch tatsächlich ausgeführte Schadsoftware.

Es werden drei Einzelverfahren bewertet. *Wazuh-Standardregeln* entscheiden positiv, wenn mindestens eine Alarmmeldung der festgelegten Regelbasis eindeutig zugeordnet ist. Das *lexikalische Punktemodell* entscheidet positiv, wenn sein Punktwert den ausschließlich auf der jeweiligen Trainingsteilstichprobe gewählten Schwellenwert erreicht. Die *statistische Vergleichsbasis* lernt Zeichen-*n*-Gramm-Gewichte ausschließlich aus derselben Trainingsteilstichprobe und verwendet die feste Wahrscheinlichkeitsgrenze 0,5. ODER- und UND-Verknüpfung von Wazuh und Punktemodell werden ergänzend ausgewertet; sie sind keine neu trainierten Verfahren.

Die fünffache Kreuzvalidierung hält alle Varianten einer Verhaltensfamilie in derselben Teilstichprobe; der grundsätzliche Einsatz von Kreuzvalidierung zur Schätzung von Modellgüte folgt Kohavi [24]. Vokabular, TF-IDF-Gewichte, Regressionsgewichte und Punktschwellenwert entstehen damit ohne Zugriff auf die jeweilige Testteilstichprobe. Eine zusätzliche Schwellenwertwahl auf allen 300 Testfällen dokumentiert nur einen festen Einsatzwert und zählt nicht als unabhängige Leistungsschätzung. Die Wazuh-Regeln bleiben unverändert; eine nachträgliche Analyse mehrerer Mindestregelstufen beschreibt lediglich alternative Betriebspunkte.

4.3 Reproduzierbarkeit und Sicherheitsanforderungen

Neutrale IDs von PST-001 bis PST-300 kodieren weder Label noch Gruppe. Der Marker `WAZUH_SAMPLE <ID>` dient ausschließlich der Zuordnung und wird vor beiden lexikalischen Auswertungen entfernt. Versionsangaben, Parameter, Quellenstände, Ergebnisdateien und Prüfsummen dokumentieren den Messlauf.

Das Ausführungsskript analysiert jeden `ScriptBlock` vorab mit dem abstrakten Syntaxbaum von PowerShell. Für positive Testfälle ist ausschließlich `Write-Output` als

aufgerufener Befehl zugelassen. Verdächtig wirkende Befehlsfolgen bleiben Zeichenketten; Base64-Beispiele werden nur zu Text dekodiert. Externe C2-Verbindungen, Persistenzänderungen, Schutzfunktionsänderungen und das Nachladen von Nutzlasten sind ausgeschlossen.

4.4 Lab-Architektur

Der Aufbau besteht aus einem Windows-11-Host und einem Wazuh-Manager unter Kali Linux im Windows Subsystem for Linux (WSL) auf derselben physischen Maschine. Ein lokaler Erfasser liest Event-ID 4104 fortlaufend aus **Microsoft-Windows-PowerShell/Operational** und schreibt den vollständigen Feldwert **ScriptBlockText** in eine JSONL-Datei. Diese Datei ist die einzige Texteingabe der beiden lexikalischen Verfahren.

Parallel überträgt der Wazuh-Agent denselben Ereigniskanal an den Manager. Das managerseitige Ereignisarchiv belegt den Empfang aller Testfälle; es dient nicht als lexikalische Eingabe. Nur die separat erzeugten Wazuh-Alarmmeldungen bestimmen die regelbasierte Entscheidung. Abbildung 4.1 zeigt diese getrennten Rollen.

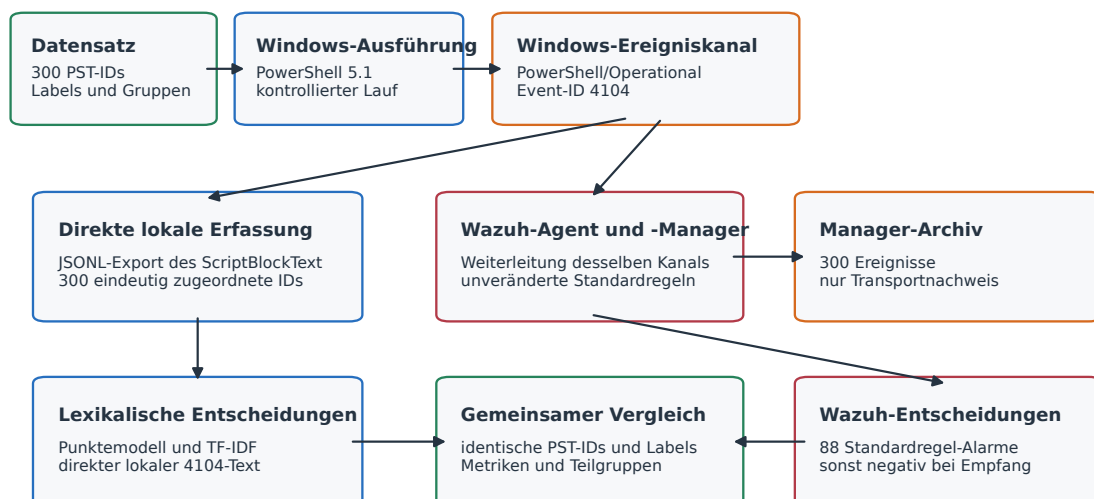


Abbildung 4.1: Laborarchitektur mit getrennten Erkennungs- und Nachweispfaden

Tabelle 4.2 dokumentiert die für den Messlauf maßgeblichen Komponenten.

Tabelle 4.2: Technische Testumgebung und verwendete Versionen

Komponente	Version beziehungsweise Ausprägung
Windows-Testsystem	Windows 11 Home 25H2, OS-Build 26200.8737, 64-Bit auf x64-Prozessor; GeräteName MSI.
PowerShell	Windows PowerShell 5.1.26100.8737.
Wazuh-Agent	Wazuh Agent 4.14.4; Agent-ID 001; Name <code>Windows-Host</code> .
Wazuh-Manager	Wazuh Server 4.14.5-rc1 unter WSL Kali; installierte PowerShell-Standardregeln.
Aufbau	Windows-Agent auf dem Hostsystem und Wazuh-Manager in WSL; kein Docker-Einsatz im Messlauf.
Python-Umgebung	Python 3.13.12; NumPy 2.3.5; SciPy 1.16.3; Matplotlib 3.10.7.
Hardware	Intel Core i7-1280P und 15,71 GB nutzbarer Arbeitsspeicher.
Datensatz	300 Testfälle; SHA-256 <code>12f6aa41...b58c4ae3</code> .
Messfenster	17.07.2026, 09:35:23 bis 09:37:52 UTC.

4.5 Datensatzdesign

Der UTF-8-kodierte JSON-Datensatz enthält für jeden Testfall unter anderem ID, Label, Datensatzgruppe, Verhaltens- und Evaluierungsgruppe, MITRE-ATT&CK-Einordnung, Obfuskierungsart, Referenztext, instrumentierten ScriptBlock, Sicherheitsklasse und Quellenreferenzen. Das Feld `analysis_text` dient der Validierung und Dokumentation. Beide lexikalischen Verfahren analysieren im Bericht ausschließlich den direkt erfassten Ereignistext. ID, Szenarioname und Instrumentierung werden vor der Merkmalsextraktion entfernt.

Der Datensatz ist synthetisch. Die 150 benignen Testfälle wurden im Rahmen der Arbeit aus typischen administrativen Aufgaben und gezielt schwierigen Grenzfällen zusammengestellt. Sie stammen nicht aus produktiven Unternehmensprotokollen oder einem realen externen Datensatz.

Die 30 positiven Grundmuster sind direkt im Datensatzgenerator definiert. MITRE ATT&CK, Atomic Red Team, Sigma, Living-off-the-Land Binaries and Scripts (LOLBAS), die Wazuh-Regelbasis und dokumentierte PowerShell-Muster dienen

ausschließlich zur Auswahl und fachlichen Einordnung der dargestellten Techniken [5, 9, 10, 11, 4]. Es wurden keine vollständigen Skripte, Payload-Dateien, C2-Implantate oder ausführbaren Schadprogramme aus diesen Quellen importiert. Der Generator erzeugt aus jedem Grundmuster eine klare Variante sowie vier obfuskierte, inerte Varianten. Die Obfuskationskategorien orientieren sich an Invoke-Obfuscation [12]. Das Quellenmanifest dokumentiert die konzeptionellen Referenzen, Abrufdaten und Repository-Stände. Es weist keine direkte Herkunft einzelner Sample-Texte nach.

Abbildung 4.2 fasst die Konstruktion und die Rolle der Quellen zusammen.

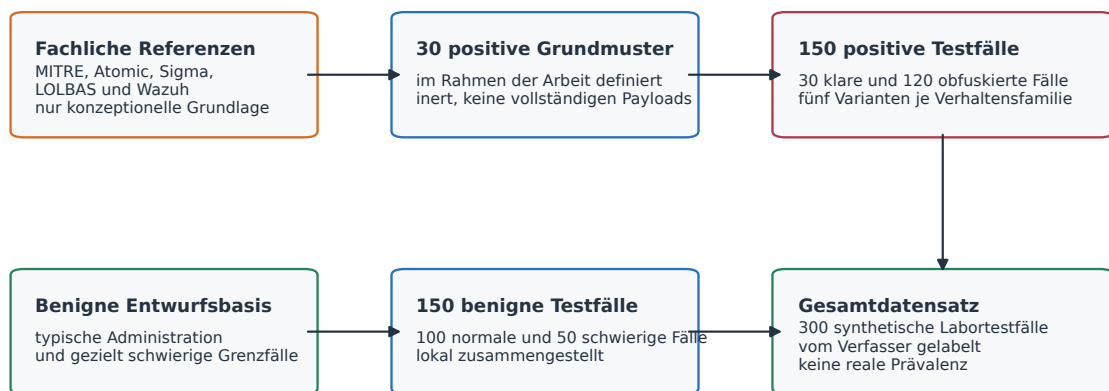


Abbildung 4.2: Konstruktion des synthetischen Labordatensatzes und Rolle der Quellen

Die Klassen umfassen jeweils 150 Testfälle und bilden daher keine reale Prävalenz ab. Die 50 schwer abzugrenzenden benignen Fälle enthalten absichtlich lexikalisch auffällige, aber legitime Konstrukte. Insgesamt bestehen 105 Evaluierungsgruppen. Die Klassenlabels und Sicherheitsbewertungen wurden vom Verfasser anhand der Konstruktion und Quellen vergeben; eine unabhängige Zweitannotation fand nicht statt. Ebenso fehlen organisationsspezifische Skripte und ein zeitlich oder institutionell getrennter externer Datensatz. Diese Punkte begrenzen die Übertragbarkeit.

Der Datensatzgenerator prüft Anzahl, eindeutige Texte, Quellenfelder, neutrale IDs und Sicherheitsmetadaten. Klassenabhängige Gerüstbestandteile werden vor der statistischen Merkmalsextraktion neutralisiert. Das Schema steht in Anhang A; zentrale Prüfsummen stehen in Anhang B.

4.6 Klassen und Testfälle

Der Datensatz umfasst 300 Testfälle mit ausgeglichener Klassenverteilung. Die positive Klasse enthält ausschließlich simulierte, nicht schädlich ausführbare Testfälle mit definierten Angriffsindikatoren. Tabelle 4.3 zeigt die vier Datensatzgruppen; Tabelle 4.4 gliedert die 120 obfuskierten positiven Fälle.

Tabelle 4.3: Klassen und Gruppen des Datensatzes

Gruppe	Label	Anzahl	Beschreibung
Normale benigne Testfälle	<code>benign</code>	100	Administrative Prozess-, Dienst-, Ereignisprotokoll- und Systemabfragen.
Schwer abzugrenzende benigne Testfälle	<code>benign</code>	50	Legitime Skripte mit längeren Zeichenketten, internen URLs, Formatierung oder JSON-, CSV-, XML- und Base64-Verarbeitung.
Klare Indikator-Testfälle	<code>simulated_malicious</code>	30	Je ein nicht ausgeführtes Grundmuster aus 30 Verhaltensfamilien.
Obfuskierte Indikator-Testfälle	<code>simulated_malicious</code>	120	Varianten der Grundmuster aus sechs Obfuskiertungskategorien.

Tabelle 4.4: Obfuskierungsarten im Datensatz

Obfuskierungsart	Anzahl	Untersuchungsziel
Base64-Dekodierung	20	Wiedergewinnung kodierter Indikatorzeichenketten.
String-Konkatenation	20	Erkennung aufgeteilter Tokens und Konkatenationsoperatoren.
Backticks	20	Umgang mit PowerShell-spezifischer Tokenunterbrechung.
Gemischte Schreibweise	20	Robustheit gegenüber wechselnder Groß- und Kleinschreibung.
Alias- und Befehlsrekonstruktion	20	Erkennung von Aliasnutzung und rekonstruierten Befehlen.
Kombinierte Obfuskation	20	Wirkung mehrerer gleichzeitiger Obfuskationstechniken.

4.7 Sicherheitskonzept der simulierten Testfälle

Netzwerkziele verwenden reservierte `.invalid`-Domänen nach RFC 2606 oder die TEST-NET-3-Adresse 203.0.113.77 nach RFC 5737 [22, 23]. Verdächtige Befehlsfolgen werden als Zeichenketten ausgegeben und nicht aufgerufen. Base64-Beispiele dekodieren ausschließlich zu Text. Das Ausführungsskript verlangt einen ausdrücklichen Laborparameter und bricht bei nicht zugelassenen AST-Befehlen ab.

Diese Konstruktion ermöglicht die wiederholbare Untersuchung textbasierter Indikatoren. Sie bildet jedoch weder die Laufzeitsemantik realer Schadsoftware noch mehrstufige Nachladevorgänge, Umgebungsprüfungen oder adaptive Umgehung ab.

4.8 Lexikalische Merkmale und Punktmodell

Das Punktmodell berechnet deterministische Merkmale des direkt erfassten `ScriptBlockText`. Dazu gehören Länge, Token- und Literalanzahl, Shannon-Entropie, Zeichenklassenanteile, Base64-Kandidaten, Backticks, Konkatenations- und Formatoperatoren, auffällige Schreibweise sowie Treffer in zehn dokumentierten

Indikator-kategorien. Entropie und Zeichenanteile werden ausgegeben, fließen jedoch nicht direkt in den Punktwert ein.

Vor der Bewertung entfernt die Pipeline ausschließlich die neutrale Testfallinstrumentierung. Sie erzeugt danach begrenzte, nicht ausgeführte Textansichten: Backticks werden normalisiert, benachbarte Stringlitterale zusammengefügt und formal gültige Base64-Kandidaten als UTF-8 oder UTF-16LE dekodiert. Der rekonstruierte Inhalt wird nie aufgerufen. Tabelle 4.5 dokumentiert die Punktregeln. Ein Testfall gilt als positiv, wenn sein Punktwert mindestens den Schwellenwert t erreicht.

Tabelle 4.5: Punktregeln des lexikalischen Detektors

Merkmalsgruppe	Bedingung	Punkte
Indikator-kategorien	Ausführung, Herunterladen, Netzwerk oder C2, Schutzumgehung, Persistenz, Zugriff auf Anmeldedaten oder LOLBin.	je +2
Weitere Kategorien	Datensammlung, Fernausführung oder Systemerkundung.	je +1
Mehrere Hochrisikokategorien	Treffer in mindestens zwei Hochrisikokategorien.	+2
Wiedergewonnener Indikator	Zusätzlicher Treffer nach Base64-Dekodierung oder Fragmentzusammenführung.	je +2
Obfuskationssignale	Base64-Kandidat, mindestens zwei Backticks, Stringrekonstruktion, mindestens vier kurze Litterale, auffällige Schreibweise oder Format- und Zeichenkonstruktion.	je +1
Kombinierte Obfuskation	Mindestens zwei unterschiedliche Obfuskationssignale.	+2
Korrektur für Systemerkundung	Nur Systemerkundung, keine Hochrisikokategorie und kein Obfuskationssignal.	-1

4.9 Statistische lexikalische Vergleichsbasis

Die statistische Vergleichsbasis verarbeitet denselben de-instrumentierten Ereignistext wie das Punktemodell. Vor der Merkmalsbildung werden Groß- und Kleinschreibung

vereinheitlicht, Variablennamen auf einen neutralen Platzhalter abgebildet sowie Zuweisungsoperatoren und terminale **Write-Output**-Variablenwrapper entfernt. Diese klassenneutrale Normalisierung verhindert, dass Namen aus dem Erzeugungsgerüst als Labelhinweis dienen; die zugewiesenen Zeichenketten bleiben erhalten.

Aus jedem Trainingstext werden Zeichen- n -Gramme der Längen drei bis fünf gebildet. Ein Fragment muss in mindestens zwei Trainingsdokumenten vorkommen; pro Teilstichprobe werden höchstens 15.000 Fragmente verwendet. Die inverse Dokumenthäufigkeit lautet

$$\text{idf}(g) = \log \left(\frac{1 + N}{1 + \text{df}(g)} \right) + 1.$$

Mit der Termhäufigkeit gewichtete Vektoren werden auf euklidische Länge eins normiert. Eine logistische Regression mit L2-Regularisierung $\lambda = 1$ wird mit dem Limited-memory-Broyden-Fletcher-Goldfarb-Shanno-Verfahren (L-BFGS) in höchstens 250 Iterationen angepasst. Die feste Entscheidungsgrenze beträgt 0,5.

Vokabular, Dokumenthäufigkeiten und Modellparameter entstehen in jeder Runde ausschließlich aus 240 Trainingstestfällen. Die 60 zurückgehaltenen Fälle werden genau einmal klassifiziert. Es findet keine Hyperparameterwahl anhand der Testteilstichproben statt. Das Verfahren ist damit eine statistische Vergleichsbasis, aber kein unabhängig extern validiertes Modell.

4.10 Wazuh-Referenzmessung und Alarmzuordnung

Die Referenzmessung verwendet ausschließlich die installierte Datei `/var/ossec/ruleset/rules/0915-win-powershell_rules.xml` mit der SHA-256-Prüfsumme `b358b3c4...ac10739f`. Eigene oder angereicherte Regeln werden nicht einbezogen. Der Alarmimport begrenzt die Auswertung auf Agent `Windows-Host`, Event-ID 4104 und die PowerShell-Standardregel-IDs 91803 bis 92000.

Eine Testfall-ID gilt nur dann als zugeordnet, wenn derselbe Zeichenkettenwert sowohl die führende Zuweisung an `$sampleId` als auch den abschließenden Marker `WAZUH_SAMPLE` enthält. Unbekannte, fehlende oder mehrere IDs führen zur Ablehnung. Der Alarmexport enthält 88 Datensätze; alle wurden eindeutig zugeordnet.

Das Alarmprotokoll enthält nur Ereignisse, die eine Regel ausgelöst haben. Deshalb wurde zusätzlich ein Ausschnitt des managerseitigen Ereignisarchivs ab dem vor dem Lauf gespeicherten Zeilenstand exportiert. `verify_manager_receipts.py` verlangt

Agent **Windows-Host**, Event-ID 4104, vollständige Instrumentierung und genau eine erwartete ID. Die Prüfung bestätigte 300 akzeptierte und 300 eindeutige IDs ohne fehlenden, doppelten oder abgelehnten Datensatz. Damit ist eine fehlende Alarmmeldung im Messfenster als negative Standardregelentscheidung interpretierbar. Der Archivexport dient ausschließlich diesem Empfangsnachweis und wird nicht von den lexikalischen Verfahren gelesen.

4.11 Evaluierungsmetriken

Die Konfusionsmatrix unterscheidet True Positives (TP), False Positives (FP), True Negatives (TN) und False Negatives (FN). Daraus werden die folgenden Kennzahlen berechnet:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP}, & \text{Recall} &= \frac{TP}{TP + FN}, \\ F_1 &= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, & \text{FPR} &= \frac{FP}{FP + TN}. \end{aligned}$$

Precision beschreibt den Anteil richtiger positiver Entscheidungen, Recall den Anteil erkannter positiver Testfälle und die False-Positive-Rate (FPR) den Anteil fälschlich positiver Entscheidungen innerhalb der negativen Klasse. Ergänzend werden Spezifität und Genauigkeit ausgegeben.

Für Wazuh, Punktemodell und die beiden Verknüpfungen werden 95%-Intervalle mit 2.000 gruppenstratifizierten Bootstrap-Wiederholungen berechnet; Resampling-Einheit ist die Evaluierungsgruppe [25]. Die statistische Vergleichsbasis wird zusätzlich für jede der fünf Testteilstichproben ausgewiesen. Weil der Datensatz ausgeglichen ist, wird die erwartete Precision außerdem für angenommene positive Prävalenzen von 0,1, 1, 5 und 50 Prozent berechnet. Precision-Recall-Maße bleiben von der Klassenverteilung abhängig [26].

4.12 Schwellenwertwahl und Kreuzvalidierung

In jeder Kreuzvalidierungsrunde wird der Schwellenwert des Punktemodells auf 240 Trainingstestfällen gewählt. Zulässig sind zunächst Werte mit einer Trainings-FPR von höchstens 0,10. Unter diesen wird nach F1-Score, danach Recall, niedrigerer FPR

und schließlich niedrigerem Schwellenwert geordnet. Der gewählte Wert wird unverändert auf die 60 zurückgehaltenen Testfälle angewendet. Die fünf Testteilstichproben ergeben zusammen genau eine Out-of-Fold-Entscheidung je Testfall.

Die gewählten Werte lauten 4, 5, 3, 4 und 6. Diese Streuung zeigt die Abhängigkeit vom jeweiligen Trainingsanteil. Auf allen 300 Testfällen ergibt dieselbe Regel den dokumentierten Einsatzwert 5; dieser Wert wird nicht zur primären Leistungsschätzung verwendet. Die FPR-Nebenbedingung gilt nur auf den Trainingsteilstichproben und garantiert keine entsprechende Test-FPR.

Die statistische Vergleichsbasis verwendet dieselben fünf Gruppenzuordnungen. Sie lernt in jeder Runde ein neues Vokabular und neue Modellgewichte, behält aber die feste Entscheidungsgrenze 0,5. Abbildung 4.3 fasst Erfassung, getrennte Entscheidungen und gemeinsame Auswertung zusammen.

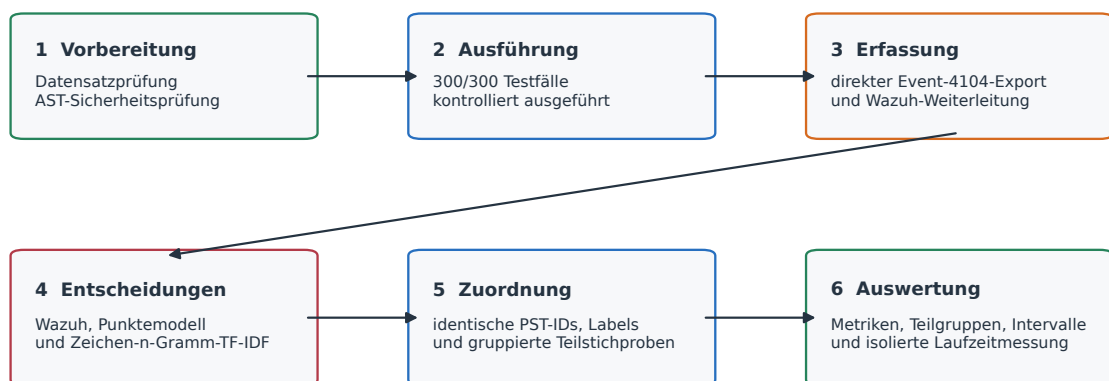


Abbildung 4.3: Versuchsablauf von der Ausführung bis zur gemeinsamen Auswertung

5 Implementierung

Dieses Kapitel beschreibt die umgesetzten Komponenten und ihre Schnittstellen. Im Mittelpunkt stehen die fortlaufende Ereigniserfassung, der Wazuh-Transport, die kontrollierte Testfallausführung sowie die beiden lexikalischen Auswertungen.

5.1 Aktivierung und Prüfung des ScriptBlock-Loggings

ScriptBlock-Logging wurde über den Richtlinienpfad der Registry aktiviert. Codeblock 5.1 zeigt die beiden administrativen PowerShell-Befehle.

Codeblock 5.1: Aktivierung des ScriptBlock-Loggings

```
1 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\  
   ScriptBlockLogging" -Force  
2 Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\  
   PowerShell\ScriptBlockLogging" -Name EnableScriptBlockLogging -  
   Value 1 -Type DWord
```

Die Prüfung erfolgt im Kanal `Microsoft-Windows-PowerShell/Operational` anhand von Event-ID 4104. Das lokale Protokoll war auf 15 MB begrenzt. Um ein Überschreiben früher Ereignisse zu vermeiden, liest `collect_event4104.ps1` neue Datensätze fortlaufend anhand der `EventRecordID` und schreibt jeden vollständigen `ScriptBlockText` unmittelbar in eine JSONL-Datei. Nach dem Lauf prüft die Pipeline Event-ID, Instrumentierung, Anzahl, Eindeutigkeit und Textgleichheit.

Abbildung 5.1 dokumentiert die Aktivierung. Abbildung 5.2 zeigt einen zugehörigen Ereigniseintrag mit neutraler Testfall-ID.

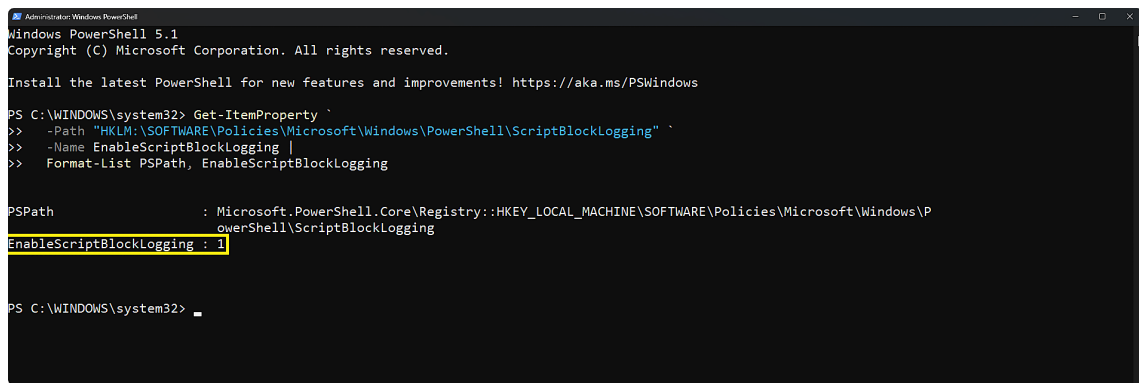


Abbildung 5.1: Aktivierter Registry-Wert für PowerShell ScriptBlock-Logging

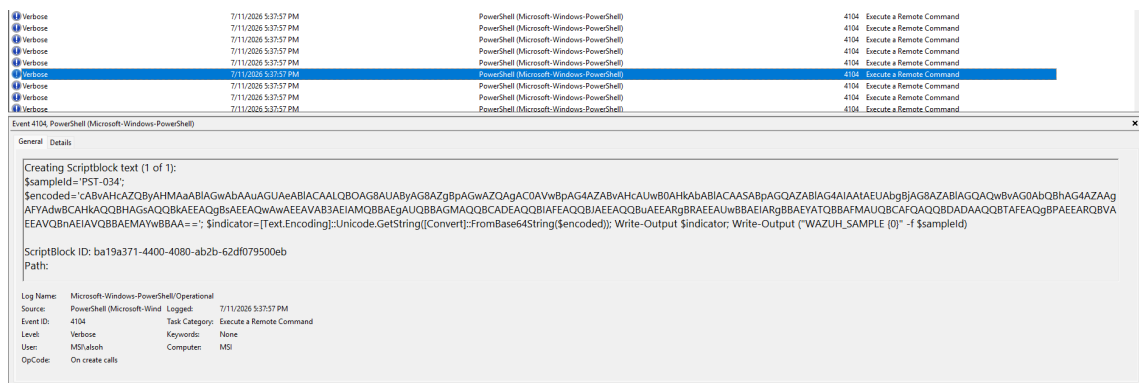


Abbildung 5.2: Event-ID 4104 mit Testfall-ID im ScriptBlockText

5.2 Wazuh-Agent und Ereigniskanal-Weiterleitung

Der Windows-Agent liest denselben PowerShell-Kanal. Der verwendete Abschnitt der Datei C:/ProgramFiles(x86)/ossec-agent/ossec.conf ist in Codeblock 5.2 dargestellt.

Codeblock 5.2: Wazuh-Agent-Konfiguration für den PowerShell-Ereigniskanal

```
1 <localfile>
2   <location>Microsoft-Windows-PowerShell/Operational</location>
3   <log_format>eventchannel</log_format>
4 </localfile>
```

Nach der Konfigurationsänderung wurde der Dienst WazuhSvc neu gestartet. Agent 001 war während des Messfensters aktiv. Die installierte Regeldatei und die Konfiguration wurden vor dem Lauf durch Prüfsummen festgehalten; lokale Wazuh-Regeln gingen nicht in die Messung ein. Abbildung 5.3 dient als ergänzender Konfigurationsnachweis.

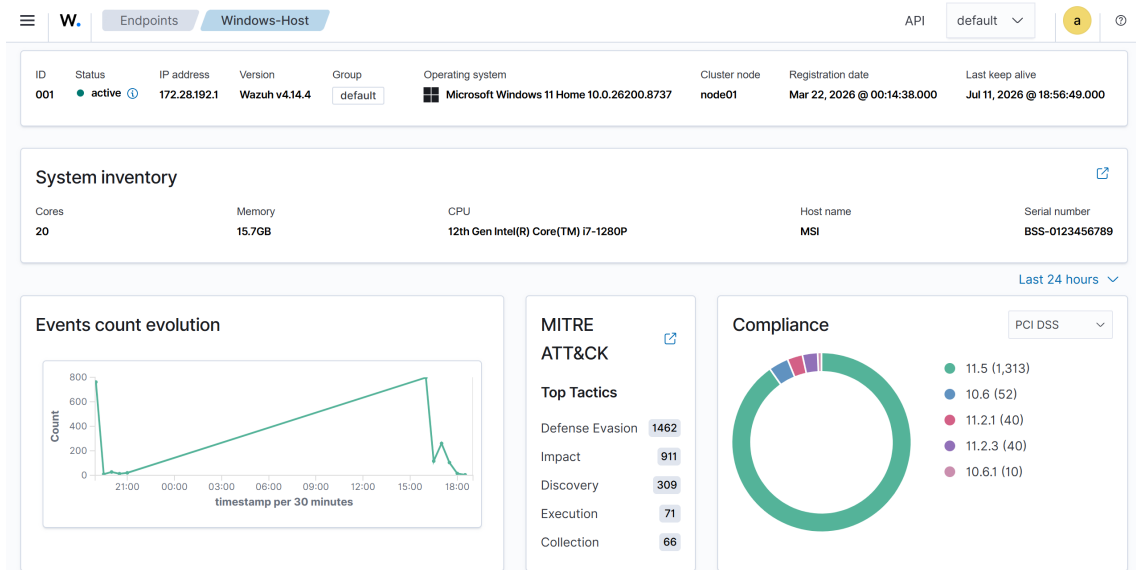


Abbildung 5.3: Aktiver Windows-Agent mit Agent-ID, Hostname und Wazuh-Version

5.3 Ausführung der Datensatz-Testfälle

Das Skript `run_dataset_samples.ps1` lädt den JSON-Datensatz und prüft vor jeder Ausführung Sicherheitsklasse und AST. Der Parameter `-IUnderstandThisIsALab` ist obligatorisch; `-PreflightOnly` führt ausschließlich die Vorprüfungen aus. Das Ausführungslog wird vor dem ersten Testfall auf Schreibbarkeit geprüft und danach für jeden Fall fortgeschrieben.

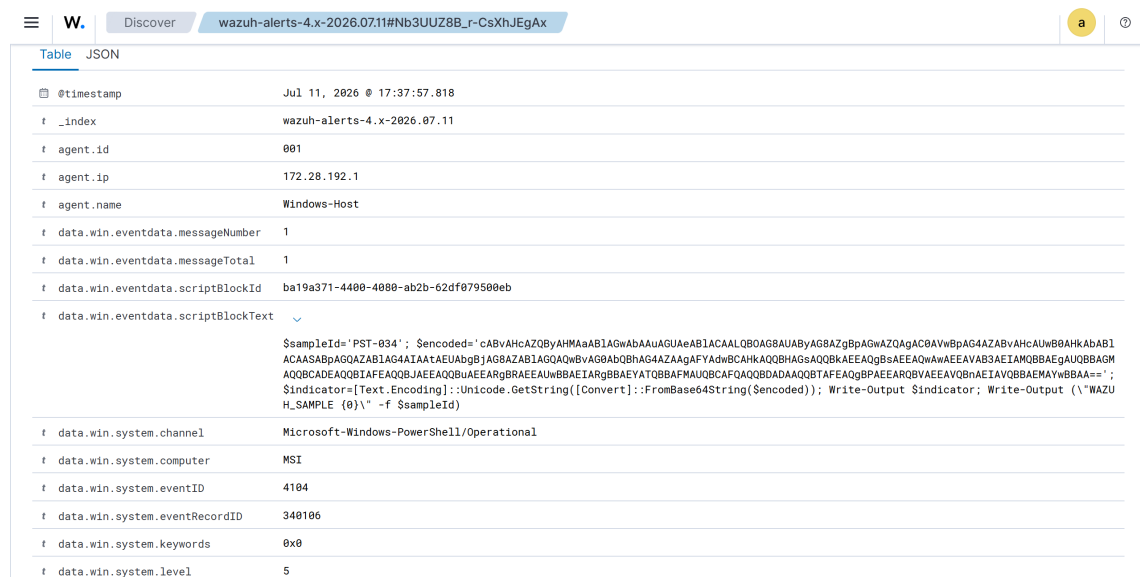
Im Messlauf betrug der Abstand zwischen zwei Testfällen 300 Millisekunden. Jeder ScriptBlock beginnt mit der eindeutigen Zuweisung an `$sampleId` und endet mit dem Marker `WAZUH_SAMPLE <ID>`. Beide Bestandteile müssen für eine gültige Zuordnung vorhanden sein.

5.4 Managerseitiger Empfangsnachweis und Alarmexport

Vor dem Lauf speichert `capture_wazuh_offsets.py` die aktuellen Zeilenstände des managerseitigen Ereignisarchivs und des Alarmprotokolls. Zusätzlich werden Prüfsummen der Agentenkonfiguration und Regeldatei erfasst. Nach dem Lauf verarbeitet

`export_wazuh_capture.py` ausschließlich die seit diesen Positionen hinzugekommenen Datensätze. Das Programm filtert Agent, Event-ID und vollständige Testfallinstrumentierung und erzeugt zwei getrennte Dateien: `manager_event4104_receipts.jsonl` für den Transportnachweis und `alerts_standard_rules_dataset_run.jsonl` für die regelbasierte Entscheidung.

`verify_manager_receipts.py` prüft den Archivausschnitt auf erwartete, fehlende und doppelte IDs. Die Ergebnisdatei `manager_receipt_summary.json` weist 300 akzeptierte Ereignisse ohne Lücke oder Duplikat aus. Der getrennte Alarmexport enthält 88 eindeutig zugeordnete Alarmmeldungen aus dem Standardregelbereich. Abbildung 5.4 zeigt einen Alarm mit Regel- und Testfallbezug; Abbildung 5.5 dokumentiert die maschinenlesbare Darstellung.



Field	Value
@timestamp	Jul 11, 2026 @ 17:37:57.818
_index	wazuh-alerts-4.x-2026.07.11
agent.id	001
agent.ip	172.28.192.1
agent.name	Windows-Host
data.win.eventdata.messageNumber	1
data.win.eventdata.messageTotal	1
data.win.eventdata.scriptBlockId	ba19a371-4400-4080-ab2b-62df079500eb
data.win.eventdata.scriptBlockText	<pre>\$sampleId='PST-034'; \$encoded='cABvAHcAZQByAHMAaB1AGwAbAAuAGUaeAB1ACAALQBOAG8AUByAG8AZgBpAGwAZQAgAC0AVwBpAG4AZABvAHcAUwB0AHKAbAB1ACAASABpAGQAZAB1AG4IAAIAEUAdgBjAG8AZAB1AGQAwBvAG8ABQbHAG4AZAAgAFYAdwBCAHKAQbHAGsAQbKAEAAQgBsAEAAQwAwAEAAVAB3AEIAMQBBAEgAUQBBAQM AQbCADEAQbIAFEAQbJAEEAQbUAEEARGBRAEEAUWBBAETARgBBAEYATQBBAFMAUQBBAFQQAQbDADAQAQbTAFEAAQgBPAAEEARQBVAEEAVQBnAETIAVQBBAEMAYWBBA=='; \$indicator=[Text.Encoding]::Unicode.GetString([Convert]::FromBase64String(\$encoded)); Write-Output \$indicator; Write-Output ("WAZU H_SAMPLE {0}" -f \$sampleId)</pre>
data.win.system.channel	Microsoft-Windows-PowerShell/Operational
data.win.system.computer	MSI
data.win.system.eventID	4104
data.win.system.eventRecordID	340106
data.win.system.keywords	0x0
data.win.system.level	5

Abbildung 5.4: Wazuh-Alarmmeldung mit Regel-ID, Event-ID 4104 und Testfall-ID

W.	Discover	wazuh-alerts-4.x-2026.07.11#Nb3UUZ8B_r-CsXhJEgAx	a	?
Table	JSON			
		<pre> { "_index": "wazuh-alerts-4.x-2026.07.11", "_id": "Nb3UUZ8B_r-CsXhJEgAx", "_score": 1, "_source": { "agent": { "ip": "172.28.192.1", "name": "Windows-Host", "id": "001" }, "manager": { "name": "MSI" }, "data": { "win": { "eventdata": { "messageNumber": "1", "messageTotal": "1", "scriptBlockText": "\$sampleId='PST-034'; \$encoded='cABvAhcAZQByAHMAaB1AGwAbAAuAGUeAB1ACAALQB0AG8AUABYAG8AZgBpAGwAZQAgAC0AVwBpAG4AZABvAHcAUwB0AHkAbAB1ACAASABpAGQAZAB1AG4IAIAAEUAbgBJAG8AZAB1AGQAOwBvAG0AbQBhAG4AZAAGAFYAdwBCHkAQQBhAGsAQQBkAEAAQgBsAEAAQwAwAEAAVAB3AEIAAQBBAGsAQBBAGMAQQBCADEAQBBIAFEAQBBJAEEAQBBuAEAAQgBRAEEAUwBBAEIARgBBAEYATQBFAUQBFAQQAQQBDADAQQBTAFEAQgBPAAEEARQBVAEEAVQBNAEIAVQBBAEMAYwBBAA='; \$indicator=[Text.Encoding]::Unicode.GetString([Convert]::FromBase64String(\$encoded)); Write-Output \$indicator; Write-Output (\\\"WAZUH_SAMPLE {0}\\\" -f \$sampleId)", "scriptBlockId": "ba19a371-4408-4088-ab2b-62df079508eb" }, "system": { "eventId": "4104", "keywords": "0xd", "providerGuid": "{a0c1853b-5c48-4b15-8766-3cf1c58f985a}", "level": "5", "channel": "Microsoft-Windows-PowerShell/Operational", "opcode": "15", </pre>		

Abbildung 5.5: JSONL-Alarmexport mit Regelmetadaten und ScriptBlockText

5.5 Python-Pipeline

pipeline.py verwendet die Python-Standardbibliothek. Im Modus run lädt es den direkten Windows-Export aus `-event-channel-events`. Für jeden erwarteten Testfall verlangt es Event-ID 4104, vollständige Instrumentierung, eine eindeutige ID und einen mit dem ausgeführten ScriptBlock übereinstimmenden Text. Ein Rückgriff auf `analysis_text` ist nicht zulässig. Nach Entfernung der Instrumentierung berechnet das Programm Merkmale, Punktwert und gruppierte Out-of-Fold-Entscheidungen. Wazuh-Alarme werden separat importiert und erst über die Testfall-ID mit den Ergebnissen verbunden.

Abbildung 5.6 zeigt diesen Datenfluss und die getrennte Einbindung der Wazuh-Alarme.

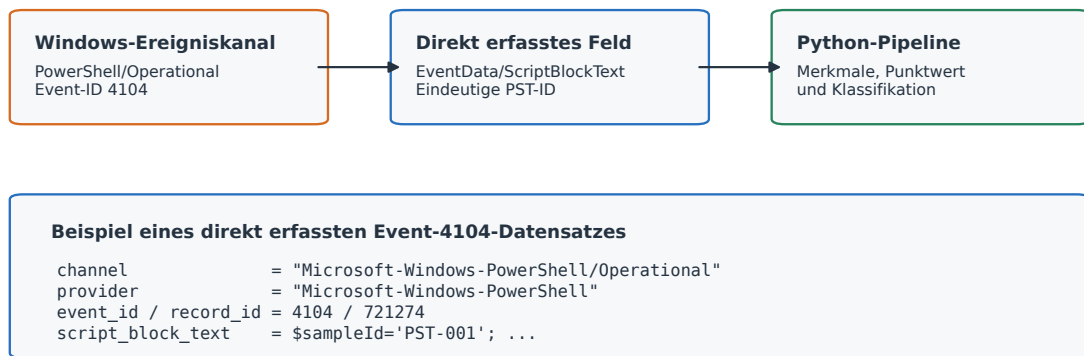


Abbildung 5.6: Direkter Datenfluss vom Windows-Ereigniskanal zu den lexikalischen Verfahren

Der folgende Aufruf reproduziert Punktemodell, Wazuh-Referenz und binäre Verknüpfungen:

Codeblock 5.3: Aufruf des zentralen Methodenvergleichs

```

1 python3 artifacts/source/pipeline.py run \
2   --dataset artifacts/dataset/powershell_scriptblock_samples.json \
3   --event-channel-events artifacts/messlauf/
   local_event4104_dataset_run.jsonl \
4   --alerts artifacts/messlauf/capture/
   alerts_standard_rules_dataset_run.jsonl \
5   --out-dir artifacts/messlauf/analysis \
6   --evaluation-mode grouped_cv --folds 5 --seed 20260711 \
7   --bootstrap-iterations 2000 --max-fpr 0.10 \
8   --agent-name Windows-Host --event-id 4104

```

`additional_analyses.py` bildet Zeichen- n -Gramm-TF-IDF-Vektoren und wertet die logistische Regression mit gruppierten Out-of-Fold-Entscheidungen aus. NumPy stellt Vektoren bereit, SciPy optimiert die Regressionsgewichte und Matplotlib erzeugt das Ergebnisdiagramm. Der Aufruf lautet:

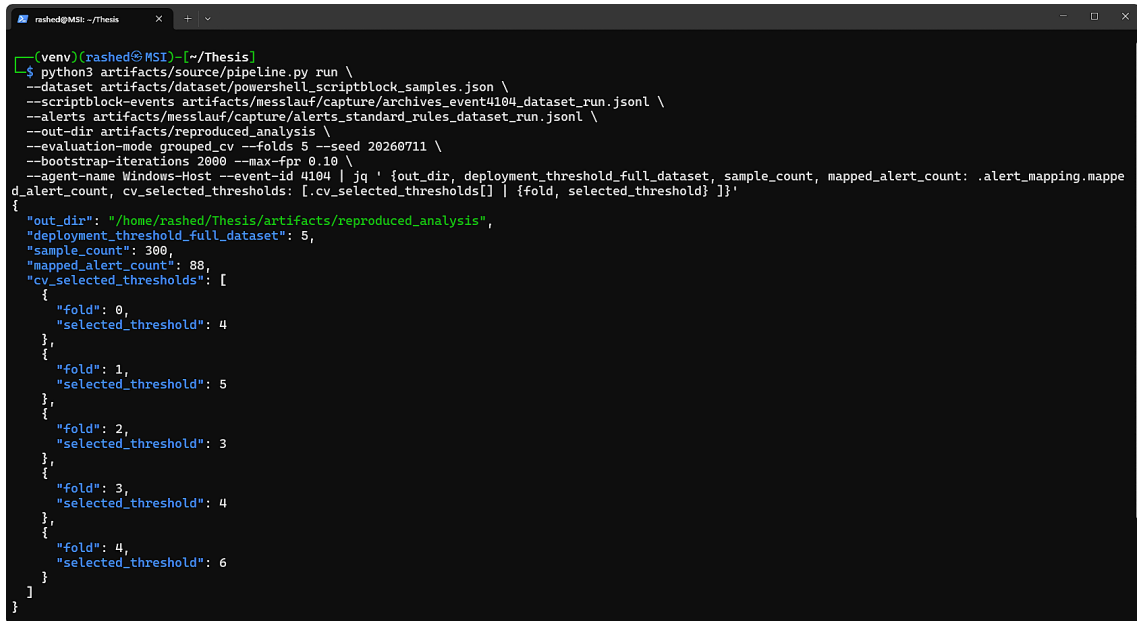
Codeblock 5.4: Aufruf der statistischen Vergleichsbasis

```

1 python3 scripts/additional_analyses.py \
2   --sample-results artifacts/messlauf/analysis/sample_results.csv \
3   --overall-metrics artifacts/messlauf/analysis/metrics_overall.csv \
4   --output-dir artifacts/messlauf/analysis \
5   --figure-output figures/baseline_comparison.pdf

```


Die erzeugten CSV- und JSON-Dateien enthalten Einzelentscheidungen, Merkmale, Schwellenwerte, Teilstichprobenergebnisse, Metriken und Laufzusammenfassungen. Abbildung 5.7 zeigt den abgeschlossenen Hauptlauf.



```
(venv)(rashed@MSI: ~/Thesis)
$ python3 artifacts/source/pipeline.py run \
--dataset artifacts/dataset/powershell_scriptblock_samples.json \
--scriptblock-events artifacts/messlauf/capture/archives_event4104_dataset_run.jsonl \
--alerts artifacts/messlauf/capture/alerts_standard_rules_dataset_run.jsonl \
--out-dir artifacts/reproduced_analysis \
--evaluation-mode grouped_cv --folds 5 --seed 20260711 \
--bootstrap-iterations 2000 --max-fpr 0.10 \
--agent-name Windows-Host --event-id 4104 | jq ' {out_dir, deployment_threshold_full_dataset, sample_count, mapped_alert_count: .alert_mapping.mappe
d_alert_count, cv_selected_thresholds: [.cv_selected_thresholds[] | {fold, selected_threshold} ]}'
{
  "out_dir": "/home/rashed/Thesis/artifacts/reproduced_analysis",
  "deployment_threshold_full_dataset": 5,
  "sample_count": 300,
  "mapped_alert_count": 88,
  "cv_selected_thresholds": [
    {
      "fold": 0,
      "selected_threshold": 4
    },
    {
      "fold": 1,
      "selected_threshold": 5
    },
    {
      "fold": 2,
      "selected_threshold": 3
    },
    {
      "fold": 3,
      "selected_threshold": 4
    },
    {
      "fold": 4,
      "selected_threshold": 6
    }
  ]
}
```

Abbildung 5.7: Abgeschlossener Pipeline-Lauf mit Eingaben und Ergebnisverzeichnis

5.6 Qualitätssicherung der Artefakte

Vierzehn automatisierte Tests prüfen die Kernpipeline. Dazu gehören Instrumentierungsentfernung, Event-Zuordnung, falsche Event-IDs, Duplikate, Textabweichungen, Base64-Dekodierung, Fragmentzusammenführung, Gruppenzuordnung und Datensatzvalidierung. Drei weitere Tests prüfen die klassenneutrale Normalisierung und Metrikbildung der statistischen Vergleichsbasis. Drei Regressionstests prüfen Offsetgrenze, Ereignisfilter und ID-Eindeutigkeit des managerseitigen Exports. Alle 20 Tests wurden erfolgreich ausgeführt.

Zusätzlich bestanden alle 300 Testfälle die AST-Vorprüfung. Der lokale Export weist 300 vollständige Ereignistexte nach, der Managerexport 300 eindeutige Empfangsbelege und der Alarmexport 88 eindeutig zugeordnete Regelalarme. Tabellen und Diagramme werden aus den erzeugten CSV-Dateien abgeleitet. Prüfsummen und Befehle stehen in Anhang B; Bildschirmaufnahmen dienen nur als ergänzende Konfigurationsnachweise.

6 Evaluation

Dieses Kapitel berichtet die Messergebnisse in der Reihenfolge der Beweiskette. Zunächst werden Vollständigkeit und Transport der Ereignisse geprüft. Danach folgen Schwellenwert, Wazuh-Regeln, statistische Vergleichsbasis, Gesamtvergleich, Teilgruppen, Fehleranalyse und Laufzeitmessung.

6.1 Datenbasis des Laborlaufs

Der Messlauf fand am 17.07.2026 von 09:35:23 bis 09:37:52 UTC statt. Das Ausführungslog enthält 300 eindeutige IDs mit dem Status `executed`. Der lokale Erfasser schrieb 300 Ereignisse mit Event-ID 4104. Es fehlen keine IDs, es existieren keine Duplikate und alle 300 `ScriptBlockText`-Werte stimmen mit den ausgeführten `ScriptBlocks` überein. Diese Texte bilden die Eingabe beider lexikalischer Verfahren.

Der managerseitige Archivausschnitt enthält ebenfalls 300 akzeptierte Ereignisse von Agent `Windows-Host`. Alle erwarteten IDs sind genau einmal vorhanden. Damit ist der Transport zum Wazuh-Manager auch für die 212 Testfälle ohne Alarm belegt. Der getrennte Alarmexport enthält 88 Standardregel-Alarme für 88 eindeutige IDs. Alle Verfahren verwenden dieselben Klassenlabels. Die Eingabe- und Ergebnisprüfsummen sind in Anhang B dokumentiert.

6.2 Schwellenwertanalyse

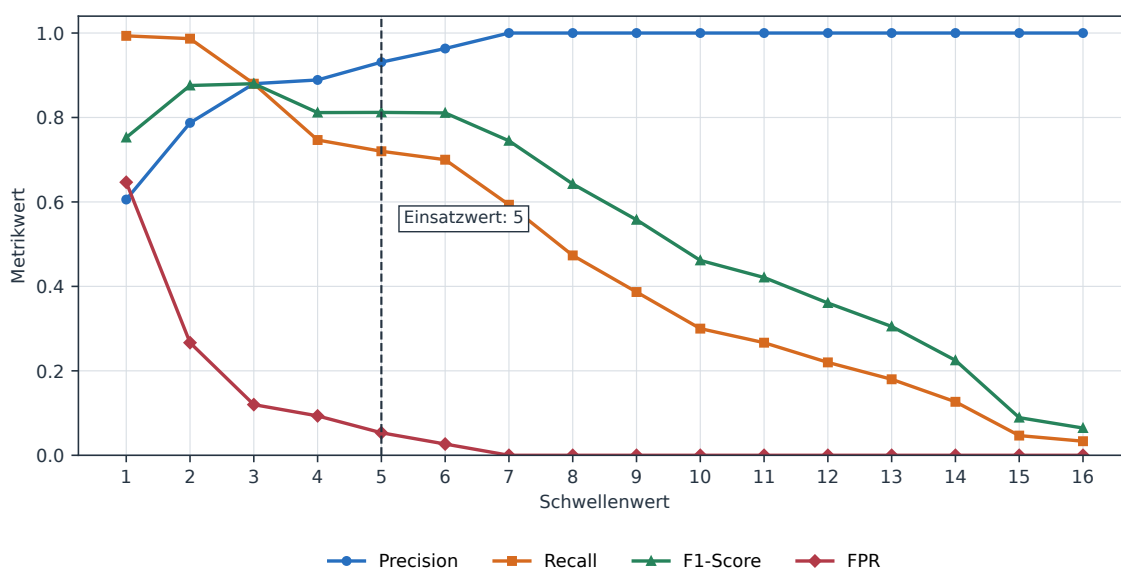
Ein niedriger Schwellenwert des Punktemodells erhöht im untersuchten Datensatz den Recall und die Zahl der False Positives. Tabelle 6.1 zeigt den entscheidungsrelevanten Bereich der Vollstichprobe. Diese Werte dienen nur zur Dokumentation des Einsatzschwellenwerts; die primäre Leistung stammt aus den Out-of-Fold-Entscheidungen.

Tabelle 6.1: Entscheidungsrelevanter Bereich der Schwellenwertanalyse

Schw.	TP	FP	TN	FN	Prec.	Rec.	F1	FPR
3	132	18	132	18	0,8800	0,8800	0,8800	0,1200
4	112	14	136	38	0,8889	0,7467	0,8116	0,0933
5	108	8	142	42	0,9310	0,7200	0,8120	0,0533
6	105	4	146	45	0,9633	0,7000	0,8108	0,0267
7	89	0	150	61	1,0000	0,5933	0,7448	0,0000

Unter der vorab festgelegten Nebenbedingung $FPR \leq 0,10$ erreicht Schwellenwert 5 auf der Vollstichprobe den höchsten F1-Score. Schwellenwert 3 hat zwar einen höheren F1-Wert, überschreitet mit 0,1200 jedoch die FPR-Grenze. Der Abstand der F1-Werte zwischen 4, 5 und 6 beträgt höchstens 0,0012; daher lässt sich aus der Vollstichprobe keine allgemeingültige Grenze ableiten.

Abbildung 6.1 stellt den Verlauf der vier Kennzahlen für alle untersuchten Schwellenwerte dar.

**Abbildung 6.1:** Kennzahlen des Punktemodells in Abhängigkeit vom Schwellenwert

6.3 Wazuh-Standardregeln

Die 88 Alarmmeldungen verteilen sich auf elf Standardregeln. Tabelle 6.2 zeigt die vollständige Häufigkeitsverteilung.

Tabelle 6.2: Ausgelöste Wazuh-Regeln im Alarmexport

Regel-ID	Anzahl	Beschreibung
91803	2	Aufruf einer Methode zur Bildschirmaufnahme.
91809	26	Mögliche Base64-Dekodierung im PowerShell-Skript.
91815	12	Prozesserkennung durch PowerShell.
91816	25	Abfrage von Systemumgebungsvariablen.
91819	4	Suche im Dateisystem.
91820	4	Rekursive Sammlung von Dateien.
91822	1	Verwendung von <code>Invoke-Command</code> für ein Subskript.
91823	2	Fernausführung mit <code>Invoke-Command</code> .
91824	1	Sammlung von Zwischenablagendaten.
91837	9	<code>Get-Content -Stream</code> oder <code>Invoke-Expression</code> .
91844	2	Möglicher Eintrag im Autostart-Registry-Pfad.

Regel 91809 erfasst Base64-Dekodierung, während die Regeln 91815, 91816 und 91819 auch administrative Tätigkeiten beschreiben. Tabelle 6.3 zeigt die Wirkung einer nachträglichen Mindestregelstufe. Ab Stufe 5 sinkt die FPR von 0,2267 auf 0,0267; zugleich sinkt der Recall von 0,3600 auf 0,1933. Die Hauptauswertung verwendet alle exportierten PowerShell-Alarme ab Stufe 3.

Tabelle 6.3: Sensitivität gegenüber der Wazuh-Mindestregelstufe

Stufe	TP	FP	TN	FN	Prec.	Rec.	F1	FPR
3	54	34	116	96	0,6136	0,3600	0,4538	0,2267
5	29	4	146	121	0,8788	0,1933	0,3169	0,0267
12	7	0	150	143	1,0000	0,0467	0,0892	0,0000

6.4 Statistische lexikalische Vergleichsbasis

Die Zeichen- n -Gramm-Vergleichsbasis erzeugt für alle 300 Testfälle Out-of-Fold-Entscheidungen. Insgesamt entstehen 135 True Positives, 14 False Positives, 136

True Negatives und 15 False Negatives. Daraus folgen eine Precision von 0,9060, ein Recall von 0,9000, ein F1-Score von 0,9030 und eine FPR von 0,0933.

Tabelle 6.4 zeigt die fünf Testteilstichproben. Ihr F1-Score liegt zwischen 0,8421 und 0,9836; die FPR liegt zwischen 0,0333 und 0,1667. Alle fünf Optimierungsläufe konvergierten nach 19 oder 20 Iterationen. Die trainierten Vokabulare enthalten zwischen 13.038 und 13.528 Zeichenfragmente.

Tabelle 6.4: Out-of-Fold-Ergebnisse der statistischen Vergleichsbasis

Teil	TP	FP	TN	FN	F1	FPR
0	30	1	29	0	0,9836	0,0333
1	24	3	27	6	0,8421	0,1000
2	30	5	25	0	0,9231	0,1667
3	26	4	26	4	0,8667	0,1333
4	25	1	29	5	0,8929	0,0333

Das Ergebnis liegt über dem festen Punktemodell, belegt jedoch keine allgemeine Überlegenheit. Beide Lexikalverfahren wurden auf demselben, vom Verfasser konstruierten Datensatz entwickelt und geprüft. Die gruppierte Kreuzvalidierung verhindert das Aufteilen einer Verhaltensfamilie, ersetzt aber keinen unabhängigen externen Testdatensatz.

6.5 Gesamtvergleich der Erkennungsverfahren

Tabelle 6.5 enthält die fünf untersuchten Entscheidungen. Abbildung 6.2 vergleicht die drei Einzelverfahren. Die statistische Vergleichsbasis erreicht im Datensatz mit 0,9030 den höchsten F1-Score. Das Punktemodell folgt mit 0,8100. Die unveränderten Wazuh-Standardregeln erreichen 0,4538.

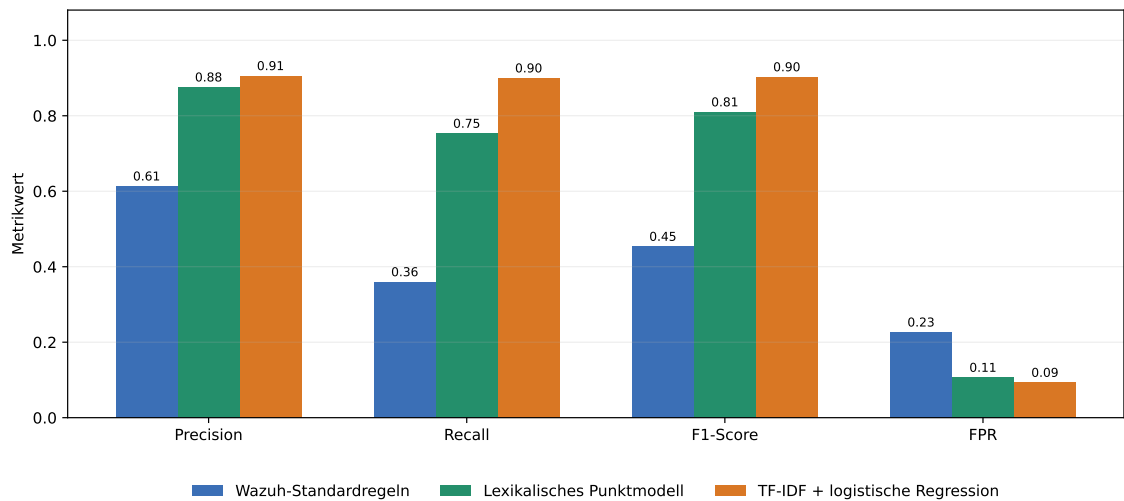


Abbildung 6.2: Vergleich der drei Einzelverfahren

Tabelle 6.5: Gesamtvergleich der Entscheidungen für 300 Testfälle

Methode	TP	FP	TN	FN	Prec.	Rec.	F1	FPR
Wazuh-Standardregeln	54	34	116	96	0,6136	0,3600	0,4538	0,2267
Lexikalisches Punktmodell	113	16	134	37	0,8760	0,7533	0,8100	0,1067
Zeichen- <i>n</i> -Gramm-TF-IDF	135	14	136	15	0,9060	0,9000	0,9030	0,0933
ODER: Wazuh/Punktmodell	127	42	108	23	0,7515	0,8467	0,7962	0,2800
UND: Wazuh/Punktmodell	40	8	142	110	0,8333	0,2667	0,4040	0,0533

Gegenüber Wazuh erhöht das Punktmodell den Recall um 0,3933 und senkt die FPR um 0,1200. Der F1-Unterschied von 0,3563 besitzt im gruppenstratifizierten Bootstrap ein 95-%-Intervall von 0,2400 bis 0,4868. Das Intervall der FPR-Differenz reicht von $-0,2267$ bis $-0,0263$. Abbildung 6.3 zeigt die Intervalle der absoluten Werte.

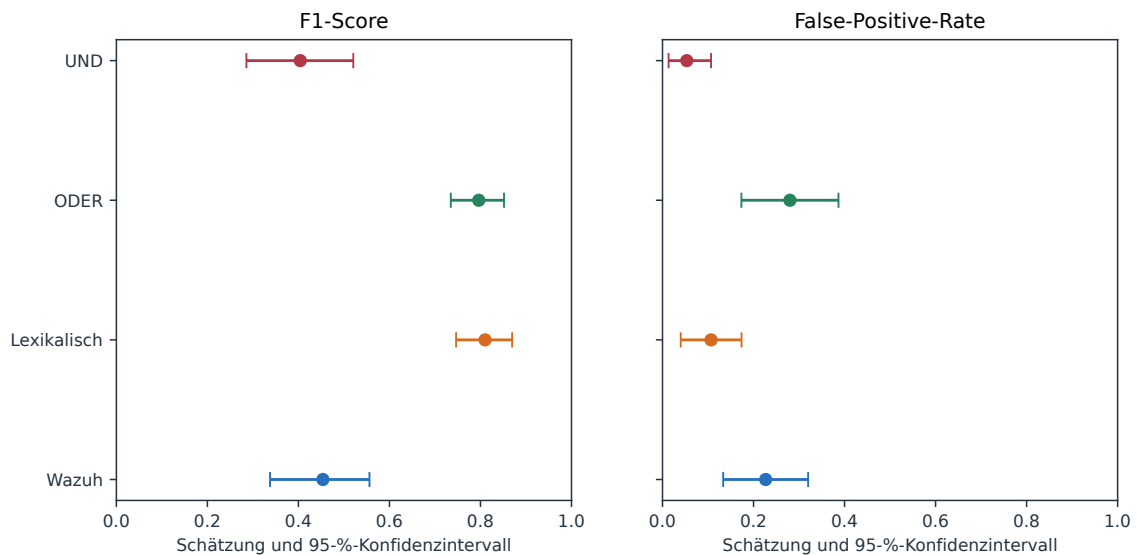


Abbildung 6.3: Gruppenstratifizierte Bootstrap-Intervalle für F1-Score und FPR

Die ODER-Verknüpfung steigert den Recall gegenüber dem Punktemodell um 0,0933, erhöht die FPR jedoch um 0,1733. Ihr F1-Unterschied beträgt $-0,0138$; das 95%-Intervall von $-0,0619$ bis $0,0379$ schließt null ein. Die UND-Verknüpfung erreicht eine FPR von 0,0533, erkennt aber nur 40 der 150 positiven Fälle.

Der Vergleich behandelt Wazuh als unveränderte Referenz, während beide lexikalischen Verfahren auf Trainingsteilstichproben angepasst werden. Die Wazuh-Stufenanalyse zeigt alternative Regelbetriebspunkte, ersetzt jedoch keine gesondert entwickelte und validierte Regelbasis. Die Tabelle vergleicht daher die dokumentierten Konfigurationen im Labor und keine allgemein optimierten Produkte.

6.6 Gruppierte Kreuzvalidierung

Tabelle 6.6 zeigt das Punktemodell in den fünf Testteilstichproben. Jede Teilstichprobe enthält 30 positive und 30 negative Testfälle. Der F1-Score liegt zwischen 0,6667 und 0,8667. Teilstichprobe 2 erreicht trotz der FPR-Bedingung im Training im Test eine FPR von 0,2000.

Tabelle 6.6: Out-of-Fold-Ergebnisse des Punktemodells

Teil	Schw.	TP	FP	TN	FN	Rec.	F1	FPR
0	4	26	4	26	4	0,8667	0,8667	0,1333
1	5	22	2	28	8	0,7333	0,8148	0,0667
2	3	27	6	24	3	0,9000	0,8571	0,2000
3	4	22	2	28	8	0,7333	0,8148	0,0667
4	6	16	2	28	14	0,5333	0,6667	0,0667

Zusammengeführt entstehen 113 True Positives und 16 False Positives. Auch die statistische Vergleichsbasis schwankt zwischen den Teilstichproben, erreicht jedoch in jeder Teilstichprobe einen F1-Score von mindestens 0,8421. Beide Auswertungen reduzieren die direkte Überschneidung verwandter Varianten, können aber weiterhin durch die Konstruktion des gemeinsamen Datensatzes beeinflusst sein.

6.7 Auswertung nach Obfuskierungsart

Innerhalb der ausschließlich positiven Obfuskierungsgruppen ist Recall die geeignete Kennzahl. Abbildung 6.4 vergleicht Wazuh und Punktemodell; Tabelle 6.7 ergänzt die statistische Vergleichsbasis.

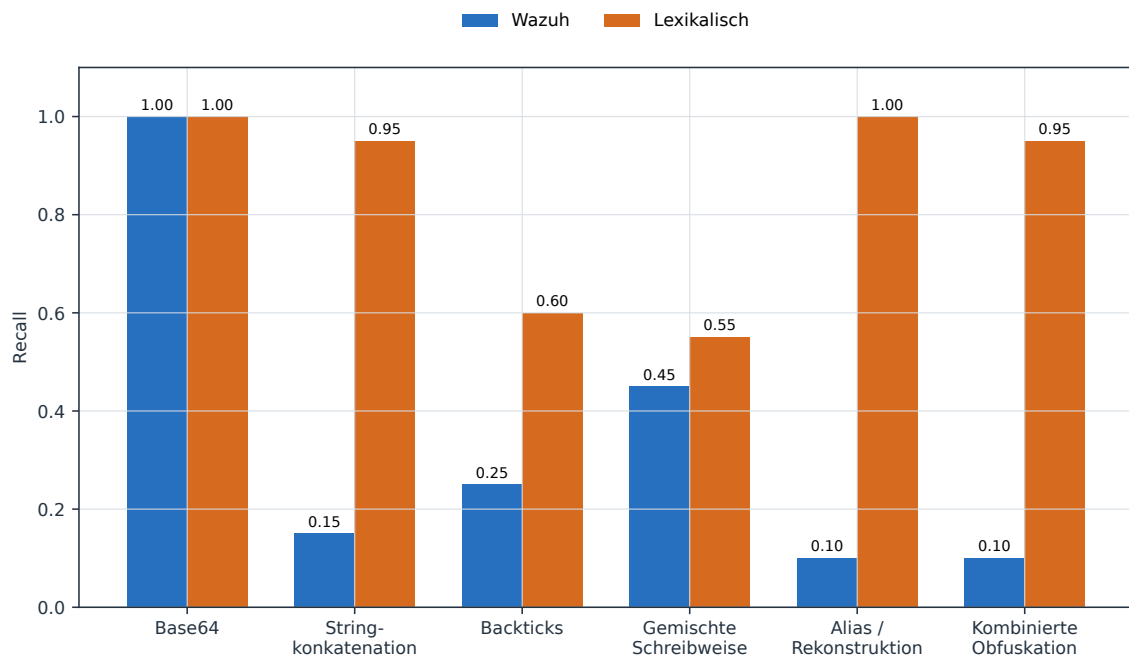


Abbildung 6.4: Recall von Wazuh und Punktemodell nach Obfuskierungsart

Tabelle 6.7: Recall der 120 obfuskerten Testfälle

Obfuskierungsart	n	Wazuh	Punktemodell	Zeichen- <i>n</i> -Gramm
Base64	20	1,0000	1,0000	1,0000
String-Konkatenation	20	0,1500	0,9500	1,0000
Backticks	20	0,2500	0,6000	0,7500
Gemischte Schreibweise	20	0,4500	0,5500	0,8500
Alias- und Befehlsrekonstruktion	20	0,1000	1,0000	1,0000
Kombinierte Obfuskation	20	0,1000	0,9500	1,0000

Alle drei Verfahren erkennen die 20 Base64-Fälle. Wazuh erreicht bei den übrigen Kategorien zwischen 0,1000 und 0,4500 Recall. Das Punktemodell erreicht 0,5500 bis 1,0000, die statistische Vergleichsbasis 0,7500 bis 1,0000. Backticks und gemischte Schreibweise weisen bei beiden Lexikalverfahren die niedrigsten Werte auf. Über die 120 obfuskerten Fälle erkennt Wazuh 41, das Punktemodell 101 und die statistische Vergleichsbasis 112.

6.8 Bewertung der False Positives

Tabelle 6.8 trennt normale und schwer abzugrenzende benigne Testfälle. Angegeben werden jeweils Anzahl der False Positives und gruppenspezifische FPR.

Tabelle 6.8: False Positives nach benigner Datensatzgruppe

Benigne Gruppe	n	Wazuh FP / FPR	Punkt FP / FPR	Zeichen- <i>n</i> - Gramm FP / FPR
Normale Administration	100	24 / 0,2400	2 / 0,0200	5 / 0,0500
Schwer abzugrenzende Fälle	50	10 / 0,2000	14 / 0,2800	9 / 0,1800
Gesamt	150	34 / 0,2267	16 / 0,1067	14 / 0,0933

Wazuh alarmiert 24 normale Verwaltungsfälle. Beim Punktemodell stammen 14 von 16 Fehllarmen aus der absichtlich anspruchsvolleren benignen Gruppe. Die statistische Vergleichsbasis verteilt ihre 14 Fehllarme auf fünf normale und neun schwer abzugrenzende Fälle. Die separate benigne Gruppe verhindert, dass nur unauffällige Verwaltungsbefehle die FPR bestimmen. Die beobachtete Precision beruht auf 50 Prozent positiven Fällen. Tabelle 6.9 überträgt die gemessenen Recall- und FPR-Werte rechnerisch auf vier angenommene Prävalenzen. Diese Rechnung setzt unveränderte bedingte Fehlerraten voraus und ist keine Messung in einer Organisation.

Tabelle 6.9: Erwartete Precision bei angenommener positiver Prävalenz

Methode	0,1 %	1 %	5 %	50 %
Wazuh-Standardregeln	0,0016	0,0158	0,0771	0,6136
Lexikalisches Punktemodell	0,0070	0,0666	0,2710	0,8760
Zeichen- <i>n</i> -Gramm-TF-IDF	0,0096	0,0888	0,3367	0,9060
ODER: Wazuh/Punktemodell	0,0030	0,0296	0,1373	0,7515
UND: Wazuh/Punktemodell	0,0050	0,0481	0,2083	0,8333

Bei einer angenommenen Prävalenz von 0,1 Prozent liegt die erwartete Precision aller Entscheidungen unter 0,01. Dieser rechnerische Befund begrenzt die betriebliche Aussage der auf dem ausgeglichenen Datensatz beobachteten Precision.

6.9 Qualitative Fehleranalyse

Tabelle 6.10 ergänzt die Kennzahlen durch vier Testfälle. Die Auswahl veranschaulicht unterschiedliche Ursachen und stellt keine zusätzliche Leistungsschätzung dar.

Tabelle 6.10: Repräsentative Einzelentscheidungen der drei Verfahren

ID	Klasse und Entscheidungen	Beobachtung	Einordnung
PST-013	Benigne; Wazuh FP, Punkt TN, statistisch TN.	Wazuh-Regel 91816; Punktwert 0.	Eine Umgebungsabfrage erfüllt das Regelmuster, enthält aber keine weiteren lexikalischen Risikosignale.
PST-021	Schwer benign; Wazuh TN, Punkt FP, statistisch FP.	Rekonstruktion von Zeichenketten und kodiert wirkender Dateipfad.	Die Textform ähnelt den positiven Obfuskationsmustern, obwohl der Zweck benign ist.
PST-001	Positiv; Wazuh FN, Punkt TP, statistisch TP.	Rekonstruierter Netzwerkindikator und mehrere Obfuskationssignale.	Beide Lexikalverfahren gewinnen aus den Fragmenten verwertbare Muster.
PST-032	Positiv; Wazuh FN, Punkt FN, statistisch TP.	Gemischte Schreibweise; Punktwert unter dem teilstichproben-spezifischen Schwellenwert.	Die Zeichenfragmente tragen zur statistischen Entscheidung bei, obwohl die festen Punktregeln nicht ausreichen.

6.10 Laufzeit des Punktemodells

Der Laufzeitversuch misst ausschließlich `extract_features` und `lexical_score` in einem Python-Prozess. Die 300 de-instrumentierten Ereignistexte werden für größere Durchläufe zyklisch wiederholt. Ereigniserfassung, JSON-Verarbeitung, Wazuh, Datei- und Netzwerkzugriffe sind ausgeschlossen. Tabelle 6.11 und Abbildung 6.5 zeigen die Ergebnisse auf der in Tabelle 4.2 beschriebenen Hardware.

Tabelle 6.11: Laufzeitmessung der Merkmalsextraktion und Punktbewertung

Anzahl	pro s	Median ms	P95 ms	P99 ms	Spitze MiB
300	601,16	1,5908	2,3498	2,5117	0,091
1.500	579,55	1,6413	2,4024	2,7794	0,128
3.000	571,50	1,6666	2,4789	2,9405	0,175

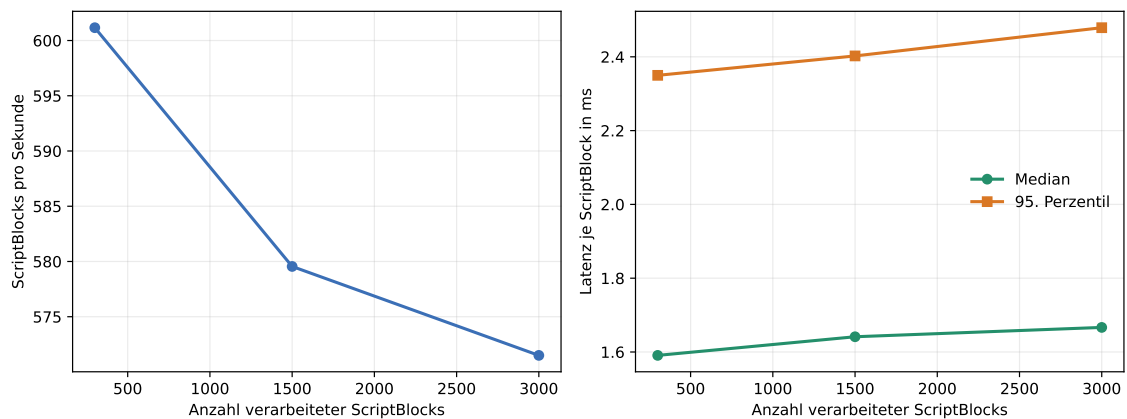


Abbildung 6.5: Durchsatz und Latenz des Punktemodells in der isolierten Laufzeitmessung

Bei 300 Auswertungen beträgt die mittlere Latenz 1,6610 Millisekunden und der Durchsatz 601,16 ScriptBlocks pro Sekunde. Bei 3.000 Auswertungen sinkt der gemessene Durchsatz auf 571,50 pro Sekunde. Der Versuch beschreibt nur die Rechenkosten des Punktemodells; er ist kein vollständiger Belastungstest vom Ereigniseingang bis zur Ausgabe.

7 Diskussion

Dieses Kapitel ordnet die Messergebnisse in Bezug auf die Forschungsfragen ein. Es trennt Aussagen, die durch den Laborlauf gestützt werden, von Grenzen, die eine Übertragung auf andere Daten oder Systeme verhindern.

7.1 Interpretation der Ergebnisse

Die drei Einzelverfahren zeigen im Datensatz unterschiedliche Fehlerprofile. Wazuh erkennt 54 der 150 positiven Fälle und erzeugt 34 Fehllarme. Das Punktemodell erkennt 113 positive Fälle bei 16 Fehllarmen. Die statistische Vergleichsbasis erkennt 135 positive Fälle bei 14 Fehllarmen. Ihr F1-Score von 0,9030 ist der höchste beobachtete Einzelwert. Die Schwankung über die fünf Teilstichproben zeigt jedoch, dass auch dieses Ergebnis vom Datenanteil abhängt.

Aus den Kennzahlen folgt nicht, dass ein Lexikalverfahren Wazuh allgemein ersetzt. Wazuh wertet Regeln und Ereigniskontext aus; die beiden anderen Verfahren klassifizieren ausschließlich den ScriptBlock-Text. Das Punktemodell wurde auf Erklärbarkeit ausgelegt, während die logistische Regression Gewichte aus dem Datensatz lernt. Der Vergleich beschreibt daher ergänzende Eigenschaften im festgelegten Labor.

7.2 Einordnung der Wazuh-Referenzmessung

Wazuh erkennt alle Base64-Fälle, weil Regel 91809 direkt auf das Dekodierungsmuster reagiert. Bei String-Konkatenation, Alias- und Befehlsrekonstruktion sowie kombinierter Obfuskation liegt der Recall dagegen bei höchstens 0,1500. Diese Transformationen verändern die Oberfläche, auf die feste Regeln reagieren.

Die 34 Fehllarme betreffen vor allem Prozess-, Umgebungs- und Dateisystemabfragen. Solche Aktivitäten können Teil eines Angriffs sein, treten aber auch in der Administration auf. Die Mindestregelstufe verschiebt den Betriebspunkt: Stufe 5 reduziert die FPR auf 0,0267, senkt den Recall jedoch auf 0,1933. Eine betriebliche Priorisierung müsste zusätzlich Regeltyp, Benutzerrolle, Host und Prozesskette berücksichtigen.

7.3 Nutzen und Grenzen der lexikalischen Verfahren

Das Punktemodell macht für jeden Testfall sichtbar, welche Kategorien und Obfuskationssignale zum Ergebnis beitragen. Begrenzte Stringrekonstruktion und Base64-Dekodierung erklären seinen Vorteil bei fragmentierten Indikatoren. Die statistische Vergleichsbasis erfasst wiederkehrende Zeichenfragmente auch dann, wenn keine feste Punktregel auslöst. Das zeigt sich bei PST-032 und im Recall der Backtick- und Mischschreibweisefälle.

Beide Verfahren analysieren nur Text. Ein hoher Punktwert oder eine hohe Modellwahrscheinlichkeit belegt keine schädliche Absicht. Legitimes Formatieren, kodierte Konfiguration und komplexe Verwaltungszeichenketten können ähnliche Muster erzeugen. Umgekehrt kann ein kurzer schädlicher Befehl ohne bekannte textuelle Form unauffällig bleiben. AST, Datenfluss, Eltern- und Kindprozesse, Benutzerkontext und Laufzeitwirkung sind keine Detektionsmerkmale.

Die ODER-Verknüpfung übernimmt Fehlalarme beider Eingangssignale; die UND-Verknüpfung verwirft zahlreiche positive Fälle. Eine weiterführende Fusion müsste Regel-ID, Wazuh-Stufe, Punktwert und Modellwahrscheinlichkeit gewichten. Eine solche Fusion wurde nicht auf dem vorliegenden Datensatz nachträglich angepasst, weil dafür ein getrennter Validierungsdatensatz erforderlich wäre.

7.4 Interne Validität

Der lokale Export bestätigt für alle 300 IDs Event-ID und exakten `ScriptBlockText`. Der managerseitige Archivausschnitt bestätigt zusätzlich den Eingang aller 300 IDs bei Wazuh. Damit ist ein Weiterleitungsverlust bei den 212 Fällen ohne Alarm für diesen Messlauf ausgeschlossen. Die 88 Alarmmeldungen wurden ebenfalls eindeutig zugeordnet.

Die gruppierte Kreuzvalidierung hält Varianten derselben Verhaltensfamilie zusammen. Beim statistischen Verfahren entstehen Vokabular, Gewichtung und Modell nur aus den jeweiligen Trainingsdaten. Klassenabhängige Namen und Ausgabewrapper des Erzeugungsgerüsts werden vorab neutralisiert. Dennoch wurden Datensatz, Labels, Punktregeln und Vorverarbeitung innerhalb derselben Arbeit entwickelt. Dadurch bleibt ein Risiko, dass beide Lexikalverfahren Besonderheiten des Korpus nutzen. Eine unabhängige Zweitannotation und ein externer Testdatensatz fehlen.

Die drei Verfahren wurden unterschiedlich vorbereitet. Wazuh bleibt unverändert. Das Punktemodell wählt seinen Schwellenwert auf den Trainingsteilstichproben, während die logistische Regression aus diesen Daten lernt. Die Wazuh-Stufenanalyse zeigt zusätzliche Betriebspunkte, ersetzt aber keine getrennt entwickelte Regelvariante. Außerdem zählt jede Wazuh-Alarmmeldung unabhängig von Regelinhalt als positive Entscheidung.

7.5 Übertragbarkeit und Grenzen der Messung

Der Datensatz enthält 300 ausgeglichene und sicher konstruierte Testfälle. Die positiven Fälle bilden textuelle Formen dokumentierter Verhaltensmuster ab. Sie enthalten jedoch weder vollständige reale Payloads noch die Laufzeitwirkung von Schadsoftware. Organisationsspezifische Skripte, zeitliche Veränderungen und eine realistische niedrige Prävalenz fehlen. Die Rechnung in Tabelle 6.9 zeigt, dass die erwartete Precision bei 0,1 Prozent positiver Prävalenz für alle Entscheidungen unter 0,01 liegt.

Der Versuch verwendet einen Windows-11-Host, Windows PowerShell 5.1, eine Wazuh-Agent- und Managerkonfiguration sowie ausschließlich Event-ID 4104. Aussagen zu PowerShell 7, anderen Betriebssystemständen, Wazuh-Versionen oder mehreren Hosts benötigen weitere Messungen. Event-ID 4104 enthält Skripttext, aber keinen vollständigen Prozess-, Benutzer- oder Netzwerkkontext. Der Laufzeitversuch misst nur Merkmalsextraktion und Punktbewertung; Erfassung, Speicherung und Alarmierung sind nicht enthalten.

7.6 Herausforderungen während der Umsetzung

Das auf 15 MB begrenzte lokale PowerShell-Protokoll konnte frühe Ereignisse überschreiben. Die fortlaufende Erfassung anhand der `EventRecordID` verhinderte dies im Messlauf. Für die Wazuh-Bewertung reichte der Alarmexport allein nicht aus, weil er negative Regelentscheidungen nicht belegt. Der zusätzliche, strikt abgegrenzte Archivausschnitt schloss diese Nachweislücke.

Auch die Testfallzuordnung erforderte eine eindeutige Instrumentierung. Eine ID wird nur akzeptiert, wenn führende Zuweisung und abschließender Marker im selben Feld vorkommen. Diese Bedingung verhindert, dass eine bloße Erwähnung einer

bekannten ID als Treffer gilt. Datenerfassung, Wazuh-Nachweis und lexikalische Analyse bleiben dabei getrennte Schritte.

7.7 Praktische Konsequenzen

Im untersuchten Labor liefern lexikalische Merkmale zusätzliche Hinweise zu obfuskierten ScriptBlocks. Vor einem betrieblichen Einsatz müssten sie jedoch mit organisationsspezifischen benignen Daten, niedriger Prävalenz und einem zeitlich getrennten Testbestand geprüft werden. Ein Alarm sollte neben dem Textsignal Prozessstartdaten, Benutzer- und Hostrolle, Sysmon-Ereignisse und Netzwerkmetadaten berücksichtigen.

Die Wazuh-Standardregeln bleiben als nachvollziehbare Referenz nützlich. Anpassungen sollten in einer gesonderten Regelvariante dokumentiert und gegen dieselben unabhängigen Testdaten bewertet werden. Das Punktemodell kann dabei ein erklärbares Zusatzsignal liefern; die statistische Vergleichsbasis zeigt, welchen weiteren Gewinn ein trainiertes Textmodell im Labor erreicht.

8 Fazit und Ausblick

Dieses Kapitel fasst die Ergebnisse zusammen, beantwortet die Forschungsfragen und bewertet die Zielerreichung. Der Ausblick benennt Arbeiten, die für eine belastbarere Übertragung erforderlich sind.

8.1 Zusammenfassung

Die Arbeit realisiert einen kontrollierten Vergleich zur Erkennung obfuskiertter PowerShell-ScriptBlocks. Alle 300 Testfälle wurden direkt aus Event-ID 4104 erfasst und textlich geprüft. Ein getrennter managerseitiger Nachweis bestätigt den Wazuh-Empfang aller 300 IDs. Der Alarmexport enthält 88 eindeutig zugeordnete Standardregel-Alarme.

Wazuh erreicht einen F1-Score von 0,4538 bei 0,2267 FPR. Das erklärbare Punktemodell erreicht 0,8100 bei 0,1067 FPR. Die statistische Zeichen- n -Gramm-Vergleichsbasis erreicht 0,9030 bei 0,0933 FPR. Alle lexikalischen Ergebnisse stammen aus nach Verhaltensfamilien gruppierten Out-of-Fold-Entscheidungen. Das Punktemodell verarbeitet in der isolierten Laufzeitmessung bei 300 Auswertungen 601,16 ScriptBlocks pro Sekunde. Diese Werte gelten nur für den untersuchten Datensatz und die beschriebene Umgebung.

8.2 Beantwortung der Forschungsfragen

F1: Welche obfuskierten Testfälle lösen eine Wazuh-Standardregel aus? Wazuh erkennt alle 20 Base64-Fälle. Der Recall beträgt 0,1500 für String-Konkatenation, 0,2500 für Backticks, 0,4500 für gemischte Schreibweise und jeweils 0,1000 für Alias- und Befehlsrekonstruktion sowie kombinierte Obfuskation. Insgesamt erkennt Wazuh 41 der 120 obfuskierten Fälle.

F2: Welche Merkmale tragen zum Punktemodell bei? Indikatorkategorien, wiedergewonnene Fragmente und mehrere gleichzeitige Obfuskationssignale liefern die höchsten Punktbeiträge. Base64-Dekodierung und Stringzusammenführung machen zuvor getrennte Indikatoren sichtbar. Entropie und Zeichenanteile werden ausgegeben, aber nicht direkt gewichtet.

F3: Wie unterscheiden sich die beiden Lexikalverfahren? Das Punktemodell erreicht 113 True Positives und 16 False Positives; jede Entscheidung lässt sich auf feste Regeln zurückführen. Die statistische Vergleichsbasis erreicht 135 True Positives und 14 False Positives. Sie erzielt den höheren F1-Score, ihre gelernten Zeichenfragmentgewichte sind jedoch weniger direkt fachlich interpretierbar.

F4: Wie wirken ODER- und UND-Verknüpfung? ODER erhöht den Recall gegenüber dem Punktemodell von 0,7533 auf 0,8467, erhöht aber die FPR von 0,1067 auf 0,2800. UND senkt die FPR auf 0,0533 und den Recall auf 0,2667. Keine der beiden Verknüpfungen erhöht den F1-Score gegenüber dem Punktemodell.

F5: Wo entstehen Fehlentscheidungen? Wazuh erzeugt 24 seiner 34 Fehllalarme bei normaler Administration. Beim Punktemodell stammen 14 von 16 Fehllalarmen aus den schwer abzugrenzenden benignen Fällen. Die statistische Vergleichsbasis erzeugt dort 9 ihrer 14 Fehllalarme. Backticks und gemischte Schreibweise weisen bei beiden Lexikalverfahren den niedrigsten Recall unter den sechs Obfuskierungsgruppen auf.

8.3 Bewertung der Zielerreichung

Die sechs Ziele aus Kapitel 1 wurden für den festgelegten Laborrahmen umgesetzt. ScriptBlock-Logging und Wazuh wurden eingerichtet. Der synthetische Datensatz wurde aus lokal definierten Testfällen erzeugt und sicher ausgeführt. Der lokale und der managerseitige Telemetrienachweis sind vollständig. Die unveränderte Wazuh-Regelbasis, das Punktemodell und die statistische Vergleichsbasis wurden mit denselben Testfällen ausgewertet. Metriken, Prävalenzrechnung, Teilgruppenanalyse, Tests, Laufzeitmessung und Prüfsummen liegen als Artefakte vor.

Nicht erreicht und nicht beansprucht wird eine allgemeine oder betriebliche Validierung. Dafür fehlen unabhängige Daten, Zweitlabels, mehrere Systeme und ein vollständiger Belastungstest vom Ereigniseingang bis zur Ausgabe.

8.4 Ausblick

Eine weitere Untersuchung sollte einen zeitlich und organisatorisch getrennten Datensatz verwenden. Benigne Daten sollten mehrere Rollen und längere Betriebszeiträume abdecken; positive Labels sollten unabhängig geprüft werden. Eine angereicherte

Wazuh-Regelbasis wäre als eigener Versuchsarm gegen unveränderte Standardregeln, Punktemodell und statistische Vergleichsbasis zu messen.

AST- und Datenflussmerkmale sowie Prozess-, Benutzer- und Netzwerkkontext könnten Fehlentscheidungen reduzieren. Zusätzlich kann der direkte Event-4104-Eingang zu einem eigenständigen Windows-Dienst weiterentwickelt werden. Dieser Dienst könnte den Kanal fortlaufend abonnieren, lokale Ergebnisse speichern und sie später an ein Überwachungs- oder Benachrichtigungssystem weitergeben. Weitere Messungen sollten PowerShell 7, mehrere Windows-Hosts und andere Wazuh-Versionen einbeziehen.

Literaturverzeichnis

- [1] Microsoft. *about_Logging_Windows*. Microsoft Learn, aktualisiert am 18.01.2026. Online verfügbar: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging_windows?view=powershell-7.6; abgerufen am 17.07.2026.
- [2] Microsoft. *about_Logging*. Microsoft Learn, aktualisiert am 29.09.2025. Online verfügbar: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging?view=powershell-5.1; abgerufen am 17.07.2026.
- [3] Wazuh Inc. *Configuring log collection for different operating systems*. Wazuh Documentation. Online verfügbar: <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/configuration.html>; abgerufen am 17.07.2026.
- [4] Wazuh Inc. *Rules Syntax – Ruleset XML syntax*. Wazuh Documentation. Online verfügbar: <https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html>; abgerufen am 17.07.2026.
- [5] MITRE. *Command and Scripting Interpreter: PowerShell, Technique T1059.001*. MITRE ATT&CK. Online verfügbar: <https://attack.mitre.org/techniques/T1059/001/>; abgerufen am 17.07.2026.
- [6] Scarfone, Karen; Mell, Peter. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. NIST Special Publication 800-94, 2007. Online verfügbar: <https://csrc.nist.gov/pubs/sp/800/94/final>; abgerufen am 17.07.2026.
- [7] Shannon, Claude E. *A Mathematical Theory of Communication*. The Bell System Technical Journal, Bd. 27, 1948, S. 379–423 und 623–656. DOI: <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- [8] Salton, Gerard; Buckley, Christopher. *Term-weighting approaches in automatic text retrieval*. Information Processing & Management, Bd. 24, Nr. 5, 1988, S. 513–523. DOI: [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0).
- [9] Red Canary. *Atomic Red Team: T1059.001 PowerShell*. Commit 94153597065a5cb453168ae177f48c9d30e89edb. Online verfügbar: <https://github.com/redcanaryco/atomic-red-team/blob/94153597065a5cb453168ae177f48c9d30e89edb/atomics/T1059.001/T1059.001.md>; abgerufen am 11.07.2026.

- [10] SigmaHQ. *Sigma rules for Windows PowerShell*. Commit 282369fa76c5cd6103b055478fbaebec8530cfa5. Online verfügbar: <https://github.com/SigmaHQ/sigma/tree/282369fa76c5cd6103b055478fbaebec8530cfa5/rules/windows/powershell>; abgerufen am 11.07.2026.
- [11] LOLBAS Project. *Living Off The Land Binaries, Scripts and Libraries*. Repository-Stand a2784c79091cb282fefb68f0056a853cfafd7e3c. Online verfügbar: <https://github.com/LOLBAS-Project/LOLBAS/tree/a2784c79091cb282fefb68f0056a853cfafd7e3c>; abgerufen am 11.07.2026.
- [12] Bohannon, Daniel. *Invoke-Obfuscation*. Commit f20e7f843edd0a3a7716736e9eddff423395dd26. Online verfügbar: <https://github.com/danielbohannon/Invoke-Obfuscation/tree/f20e7f843edd0a3a7716736e9eddff423395dd26>; abgerufen am 11.07.2026.
- [13] Hendler, Danny; Kels, Shay; Rubin, Amir. *Detecting Malicious PowerShell Commands Using Deep Neural Networks*. Proceedings of the 2018 ACM Asia Conference on Computer and Communications Security, 2018, S. 187–197. DOI: <https://doi.org/10.1145/3196494.3196511>.
- [14] Rusak, Gili; Al-Dujaili, Abdullah; O'Reilly, Una-May. *AST-Based Deep Learning for Detecting Malicious PowerShell*. Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, 2018. DOI: <https://doi.org/10.1145/3243734.3278496>.
- [15] Rubin, Amir; Kels, Shay; Hendler, Danny. *AMSI-Based Detection of Malicious PowerShell Code Using Contextual Embeddings*. Proceedings of the 15th ACM Asia Conference on Computer and Communications Security, 2020. DOI: <https://doi.org/10.1145/3320269.3384742>.
- [16] Hung, Hsiang-Hua; Chen, Jiann-Liang; Ma, Yi-Wei. *Machine Learning Approaches to Malicious PowerShell Scripts Detection and Feature Combination Analysis*. Journal of Internet Technology, Bd. 25, Nr. 1, 2024, S. 167–173. DOI: <https://doi.org/10.53106/160792642024012501014>.
- [17] Wu, Min-Hao; Hsu, Fu-Hau; Hunag, Jian-Hong; Wang, Keyuan; Liu, Yen-Yu; Chen, Jian-Xin; Wang, Hao-Jyun; Yang, Hao-Tsung. *MPSD: A Robust Defense Mechanism against Malicious PowerShell Scripts in Windows Systems*. Electronics, Bd. 13, Nr. 18, 2024, Art. 3717. DOI: <https://doi.org/10.3390/electronics13183717>.

- [18] Liu, Chao; Xia, Bin; Yu, Min; Liu, Yunzheng. *PSDEM: A Feasible De-Obfuscation Method for Malicious PowerShell Detection*. 2018 IEEE Symposium on Computers and Communications, 2018, S. 825–831. DOI: <https://doi.org/10.1109/ISCC.2018.8538691>.
- [19] Ugarte, Denis; Maiorca, Davide; Cara, Fabrizio; Giacinto, Giorgio. *PowerDrive: Accurate De-Obfuscation and Analysis of PowerShell Malware*. Detection of Intrusions and Malware, and Vulnerability Assessment, Lecture Notes in Computer Science, 2019, S. 240–259. DOI: https://doi.org/10.1007/978-3-030-22038-9_12.
- [20] Li, Zhenyuan; Chen, Qi Alfred; Xiong, Chunlin; Chen, Yan; Zhu, Tiantian; Yang, Hai. *Effective and Light-Weight Deobfuscation and Semantic-Aware Attack Detection for PowerShell Scripts*. Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, S. 1831–1847. DOI: <https://doi.org/10.1145/3319535.3363187>.
- [21] Li, Ruijie; Zhang, Chenyang; Chai, Huajun; Ying, Lingyun; Duan, Haixin; Tao, Jun. *PowerPeeler: A Precise and General Dynamic Deobfuscation Method for PowerShell Scripts*. Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, 2024. DOI: <https://doi.org/10.1145/3658644.3670310>.
- [22] Eastlake, Donald E.; Panitz, Aliza R. *Reserved Top Level DNS Names*. RFC 2606, Internet Engineering Task Force, 1999. DOI: <https://doi.org/10.17487/RFC2606>.
- [23] Arkko, Jari; Cotton, Michelle; Vegoda, Leo. *IPv4 Address Blocks Reserved for Documentation*. RFC 5737, Internet Engineering Task Force, 2010. DOI: <https://doi.org/10.17487/RFC5737>.
- [24] Kohavi, Ron. *A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection*. Proceedings of the 14th International Joint Conference on Artificial Intelligence, 1995, S. 1137–1145. Online verfügbar: <https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf>; abgerufen am 17.07.2026.
- [25] Efron, Bradley. *Bootstrap Methods: Another Look at the Jackknife*. The Annals of Statistics, Bd. 7, Nr. 1, 1979, S. 1–26. DOI: <https://doi.org/10.1214/aos/1176344552>.
- [26] Saito, Takaya; Rehmsmeier, Marc. *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*.

PLOS ONE, Bd. 10, Nr. 3, 2015, e0118432. DOI: <https://doi.org/10.1371/journal.pone.0118432>.

A Datensatzstruktur

Der Datensatz ist eine UTF-8-kodierte JSON-Liste. Codeblock A.1 zeigt zentrale Felder in gekürzter Form. `analysis_text` dient als Referenz für Validierung und Dokumentation. Die berichteten lexikalischen Entscheidungen verwenden dagegen den zugeordneten, direkt aus Event-ID 4104 erfassten `ScriptBlockText`. Ergebnisfelder werden getrennt erzeugt.

Codeblock A.1: Vereinfachte Datensatzstruktur

```
1 {
2   "schema_version": 5,
3   "id": "PST-002",
4   "label": "benign",
5   "class_id": 0,
6   "dataset_group": "hard_benign",
7   "scenario": "hard_environment_expansion_v1",
8   "behavior_family": "hard_environment_expansion",
9   "evaluation_group": "benign:hard_environment_expansion",
10  "mitre_attack_id": "N/A",
11  "obfuscation_type": "none",
12  "analysis_text": "$template = 'backup_path=...'; ...",
13  "scriptblock": "$sampleId='PST-002'; ... WAZUH_SAMPLE {0} ...",
14  "safety_class": "benign_safe_execution",
15  "source_snapshot_ids": ["microsoft_powershell"]
16 }
```


B Reproduzierbarkeit und Artefakte

Die Prüfsummen in Tabelle B.1 beziehen sich auf die Dateien, aus denen die berichteten Werte erzeugt wurden. `capture_start.json`, `capture_summary.json`, `run_summary.json` und `manager_receipt_summary.json` dokumentieren Eingabepfade, Zeilenstände, Parameter und Zählwerte. Die ausführbaren Befehle stehen zusätzlich in `BUILD.md`.

Tabelle B.1: Prüfsummen der zentralen Versuchsartefakte

Artefakt	Zweck	SHA-256
Datensatz	300 gelabelte ScriptBlocks.	12F6AA415BB8FE96B50F854C8590E98769ECE2C2CBD3775118294E32B58C4AE3
Quellenmanifest	Quellen und Repository-Stände.	B311EF8B9DB1574A991667D17ED77E111738854B1E8AE1FE67AF446AAA9B0706
<code>pipeline.py</code>	Punktemodell, Alarmimport, Kreuzvalidierung und Metriken.	A4611EA73A7B63BD90C2D1068E0CD267F377F38D7CF9B3EC5707A418A3461646
Kernpipeline-Tests	Vierzehn automatisierte Prüfungen.	555EE3482B893B5207683171ABFB0DE587568D6E3816D67A90AE297591D8756B
<code>additional_analyses.py</code>	Statistische Vergleichsbasis und Diagramm.	0D3B8284308ED245A94C7B26B041E55989C2626A0B5508C67ABC5D9C62FB2513
Zusatztests	Drei Prüfungen der statistischen Auswertung.	3699406865C98121A463DF8BCD13669ECA22385619FDDBB0CB9F714FB9946600
<code>capture_wazuh_of_fsets.py</code>	Startpositionen von Archiv und Alarmprotokoll.	9802FAA118C06C06929020B78BE8D46F5ED233DD2847BC58B0543354A65D0BA7

Fortsetzung auf der nächsten Seite

Fortsetzung von Tabelle B.1

Artefakt	Zweck	SHA-256
<code>export_wazuh_capture.py</code>	Getrennter Export von Empfangsbelegen und Regelalarmen.	834F4E5422A05D6A4263453D86 24825A331DA4F86A6944D19EF3 3EC88EBOC3A5
Wazuh-Exporttests	Drei Prüfungen von Offset, Filterung und ID-Eindeutigkeit.	60AB4BF2A01DE61162AE454ECC 79F49A824C2D62CFFB4D337015 6F04BD2E6A86
<code>benchmark_pipeline.py</code>	Abgegrenzte Laufzeitmessung.	2929AD3151FFD6C150BEA9C42B 6AB606C8883BAAE500AF9C6E41 4BBBC08CF8FC
PowerShell-Ausführungsskript	AST-Prüfung und kontrollierte Ausführung.	59BBEA02C3B912174EDC389A53 5789477E908578A21326F99392 2319EFABDE62
Lokaler Event-Erfasser	Fortlaufender Export von Event-ID 4104.	3DA30EA9979896689C047B956F F6B9158B13F896F98954D3C46C DA1E5BFC679C
Manager-Prüfprogramm	Prüfung aller managerseitigen Empfangsbelege.	A255F3CC840688A39296BDD953 5A6FF4A2E3105C5B26D121DD6E D99B55BF5346
Lokaler Event-Export	300 direkt erfasste Ereignistexte.	127ADF5AED259214836D8270D8 09A45A64A46AC05F988116C58E 07EE8ABB4A90
Manager-Empfangsbelege	300 weitergeleitete Event-4104-Ereignisse.	6D12576F4EB711189989025C1C 6AB4E25DCB307579F40B334B46 8583F35A2F62
Wazuh-Alarmexport	88 Alarmmeldungen der Standardregeln.	B155CD316B3F0324DAD612CAFB DD1155068DF859594E01BB8EAA D5784EA3AFE2
Ausführungslog	Status und Zeitstempel der 300 Testfälle.	CD3EA5CCE929CFE9CF19FFFA42 86CC79EC4CC110AEC35B5B517B A5D142D4F9CF
Wazuh-Gesamtmetriken	Kennzahlen von Wazuh, Punktemodell und Verknüpfungen.	9AB55306746043F4CF016C7CC9 5563B2E7162D8D553C13D42E8F 7FC37A036E88

Fortsetzung auf der nächsten Seite

Fortsetzung von Tabelle B.1

Artefakt	Zweck	SHA-256
Statistische Metriken	Gesamtwerte der Zeichen- <i>n</i> -Gramm-Vergleichsbasis.	2C50FD65354E3509D9BC4FD50F5BBC3C85787E0AC82BFE5720D5EEF90F4CF370
Laufzeitwerte	Messreihen mit 300, 1.500 und 3.000 Auswertungen.	1B65A9DAB99A6F1811B8B275CC5A7B6548D98411C662464DC8E88636C15CCA94
Wazuh-Regeldatei	Installierte PowerShell-Standardregeln.	B358B3C4DD597FDE075E7749401FEFF78728431F9E24C77A3C01399CAC10739F

Dokumentation der verwendeten Hilfsmittel

Hilfsmittel	Verwendungszweck	Umfang und Kontrolle
LaTeX	Satz, Tabellen, Abbildungsverweise, Verzeichnisse und Literaturverzeichnis.	Der Quelltext wurde manuell geprüft und kompiliert.
Wazuh, PowerShell und Windows Event Viewer	Aufbau der Laborumgebung, Erfassung von ScriptBlock-Logs und Export der Wazuh-Alarmmeldungen.	Die Ergebnisse stammen aus dem beschriebenen Laborlauf und wurden über die Pipeline ausgewertet.
Python, NumPy, SciPy und Matplotlib	Datenaufbereitung, Merkmalsextraktion, statistische Vergleichsbasis, Metrik- und Laufzeitberechnung sowie Ergebnisdiagramme.	Berechnete Werte und Diagrammdaten wurden mit den exportierten CSV-Dateien abgeglichen.
Offizielle Dokumentationen und Fachquellen	Technische Einordnung von ScriptBlock-Logging, Wazuh, MITRE ATT&CK und Evaluierungsmetriken.	Quellen sind im Literaturverzeichnis angegeben und wurden inhaltlich geprüft.
Git und GitHub	Versionierung und Bereitstellung der projektbezogenen Quell- und Ergebnisartefakte.	Veröffentlichte Dateien wurden vor der Verlinkung geprüft.
KI-gestütztes Sprachmodell (OpenAI ChatGPT/Codex)	Unterstützung bei Struktur- und Sprachprüfung, LaTeX-Satz, Programmierung und Fehlersuche, Testentwurf, statistischer Auswertung sowie Diagrammerzeugung.	Sämtliche Vorschläge, Quelltexte, Berechnungen und Aussagen wurden durch den Verfasser anhand der Primärquellen und erzeugten Artefakte geprüft; das System wurde nicht als wissenschaftliche Quelle verwendet.

Quellcode und GitHub-Repository

Der Quellcode der entwickelten Auswertungspipeline, die LaTeX-Artefakte und die projektbezogenen Dateien sind im folgenden GitHub-Repository verfügbar:

`https://github.com/4fv7/thesis`